



中国研究生创新实践系列大赛
“华为杯”第二十一届中国研究生
数学建模竞赛

学 校 山东大学

参赛队号 24104220149

1.张良收

队员姓名 2.赵佳

3.王硕

中国研究生创新实践系列大赛
“华为杯”第二十一届中国研究生
数学建模竞赛

题 目： 基于多因素分析与深度学习的磁芯损耗建模

摘 要：

磁性元件是第三代功率半导体发展必须要研究的重点，而现有的磁芯材料损耗模型与实际应用的需求有较大差异。在此背景下，本文主要研究了磁性元件磁芯材料损耗精确计算问题；首先对磁通密度数据进行**特征提取与分析**，实现励磁波形的分类，接着使用**三次温度修正**的方法对斯坦麦茨方程进行修正，并使用**动态参数的粒子群算法**进行参数寻优；然后通过**差异性分析**探寻了温度、波形、材料如何独立并协同影响磁芯损耗；之后基于上述特征对磁芯损耗预测提出了一种**特征权重提取的 Bi-LSTM 网络**进行预测；最后，通过改进后的蚁群算法分析最小损耗与最大传输磁能的条件，利用 Python、Pytorch 等工具给出了合理预测。

针对问题一中励磁波形的识别精度不高的问题，本文首先通过绘制频率直方图和波形图，对经过预处理的磁通密度数据进行**可视化分析和快速傅里叶变换**，从中提取出相关的**时域特征和频域特征**。通过**主成分分析**，提取出反映磁通密度数据特征的主成分变量。在训练集上进行差异性分析识别最具分辨性的磁通密度波形特征。最后，本章利用**支持向量机、决策树和集成学习算法**对不同励磁波形进行了分类实验，结果表明三种分类算法**准确率都能到 100%**，证明了特征提取的有效性以及分类的准确性。

针对问题二中原始斯坦麦茨方程对不同温度预测误差较大的问题，本文首先设计了**比例、二次、三次、指数、幂级数**五种温度修正方法对斯坦麦茨方程进行修正，并且使用**动态参数调整的粒子群算法、最小二乘法、模拟退火算法**对原始斯坦麦茨方程和温度修正后的方程进行参数寻优。寻优结果表明动态参数调整的粒子群算法比常值参数效果更好，决定系数增加了 0.041，三次函数温度修正方法最好，**决定系数为 0.9955**，相较于未修正的预测决定系数 0.9448 **提升了 0.0507**。

针对问题三探寻温度、励磁波形和磁芯材料对磁芯损耗的影响问题，本文首先在**单一因素**下对磁芯损耗的**差异性**进行分析，接着，通过计算**两两因素交叉分类**下的磁芯损耗**均值和方差**，分析两因素之间的协同效应。基于以上分析结果，确定出能够使磁芯损耗最小的最佳因素组合：**90 度、正弦波、材料四**。最后，通过三因素分类下的磁芯损耗均值和方差的热点图验证所预测的最佳因素的准确性和有效性。

针对问题四中缺乏一种能够跨越不同材料类型与工况条件的磁芯损耗预测模型的问题，本文首先根据斯坦麦茨方程选取了**磁通密度最大值**为磁通密度特征，

另外选取温度、波形、材料、频率作为特征输入，并且对波形、材料进行**独热编码**，对长尾性数据的磁芯损耗做 $\ln$ 变换，设计了**特征权重提取的双向 LSTM** 网络用作磁芯损耗预测模型。结果表明设计的网络在验证集上决定系数达到了 **0.998**，优于梯度提升树的 0.995、卷积神经网络的 0.9837，并且对该模型的**泛化能力**在测试集以及加了噪声的测试集上进行测试，结果表明在加了噪声的测试集上决定系数仍达到了 **0.9952**，优于另外两种方法。

针对问题五提出的多目标优化问题，本文使用**单目标转化**和**智能优化算法**两种方式对模型进行求解。对于多目标转化单目标优化问题，本文采用遗传算法和粒子群算法对模型进行求解，通过两个算法对比分析求得最优解，最终结果显示，遗传算法求得的最优解优于粒子群算法求得的最优解，最终多目标转化单目标优化方式求得的最优解为**磁芯材料二，正弦波，温度 90，频率 79680，磁通密度峰值 0.021，磁芯损耗为 426.3647，最大传输磁能为 1673.28**。对于智能优化算法，本文通过 **NSGA-II 算法**和 **MOPSO 算法**两种优化算法分别求取了多目标优化问题的 Pareto 最优解，绘制了 Pareto 前沿，并对两个算法求取的 Pareto 最优解进行了评价分析，最终得到 NSGA-II 算法求得 Pareto 最优解优于 MOPSO 算法求得 Pareto 最优解。

关键词：磁芯损耗预测；时频域特征提取；粒子群算法；帕累托最

优；双向 LSTM 网络

目录

一、问题重述.....	5
1.1 问题背景.....	5
1.2 本文拟解决的问题.....	5
二、模型假设与符号说明.....	6
2.1 模型的基本假设.....	6
2.2 模型符号说明.....	6
三、技术路线图.....	7
四、问题一的模型的建立与求解.....	8
4.1 问题分析.....	8
4.2 模型建立与求解.....	8
4.2.1 数据分析与特征提取.....	8
4.2.2 支持向量机.....	13
4.2.3 决策树.....	15
4.2.4 集成学习.....	16
4.2.5 模型分析.....	17
4.3 本章小结.....	19
五、问题二的模型的建立与求解.....	20
5.1 问题分析.....	20
5.2 模型建立与求解.....	20
5.2.1 最小二乘法.....	20
5.2.2 模拟退火算法.....	20
5.2.3 加入动态参数调整的粒子群算法.....	21
5.2.4 原始斯坦麦茨方程参数寻优.....	22
5.2.5 加入温度修正的斯坦麦茨方程及其参数寻优.....	25
5.3 模型小结.....	28
六、问题三的模型的建立与求解.....	29
6.1 问题分析.....	29
6.2 模型建立与求解.....	29
6.2.1 数据提取与分布分析.....	29
6.2.2 单因素对磁芯损耗的影响分析.....	30
6.2.3 两两因素对磁芯损耗的协同影响分析.....	32
6.2.4 三因素协同影响下的最优条件分析及热点图.....	35
6.3 模型小结.....	36
七、问题四的模型的建立与求解.....	38
7.1 问题分析.....	38
7.2 模型建立与求解.....	38
7.2.1 双向 LSTM 模型.....	38
7.2.2 梯度提升树模型.....	39
7.2.3 数据特征提取.....	39
7.2.4 特征重要性提取的双向长短时记忆回归网络.....	41
7.3 模型小结.....	45
八、问题五的模型的建立与求解.....	46
8.1 问题分析.....	46

8.2 模型建立与求解.....	46
8.2.1 决策变量数据整合及目标函数建立.....	46
8.2.2 多目标转化单目标优化求解.....	47
8.2.3 NSGA-II 算法.....	49
8.3 本章小结.....	53
九、模型评价与推广.....	54
9.1 模型的优点.....	54
9.2 模型的缺点.....	54
9.3 模型改进与推广.....	54
十、参考文献.....	55
附 录：Python 代码.....	56

公众号：学生竞赛库

一、问题重述

1.1 问题背景

随着全球能源需求的持续增长，能源转换技术的重要性愈加突出。第三代功率半导体技术的成熟使得电力电子设备逐步向高频化、高功率密度和高效化方向发展。在这一背景下，磁性元件在电力电子系统中发挥着关键作用。磁性元件不仅承担着磁能的传递、存储和滤波等功能，其直接影响功率变换器的效率、体积、重量和成本。特别是在高频工作条件下，磁性元件的损耗问题已成为限制功率变换器效率提升的瓶颈之一。目前，磁性元件正朝着高频化、高功率、微型化和节能化的方向发展，这对降低电力电子系统的能耗和提高转化效率具有重要意义。

磁性元件的损耗主要包括磁滞损耗、涡流损耗和剩余损耗，这些损耗与温度、频率、磁通密度和励磁波形等因素密切相关，并呈现出复杂的非线性行为。然而，现有的磁芯损耗模型，如斯坦麦茨方程，虽然能够提供一定的损耗预测，但在处理不同材料、非正弦波形及温度变化时，精度明显不足。这些模型无法全面适应多变的工况，难以满足实际应用的需求，因此亟需建立适应多样化磁性材料、励磁波形、温度和频率变化的高精度磁芯损耗预测模型。

本研究的宗旨是针对高频电力电子系统中的磁芯损耗问题，系统分析励磁波形、温度及磁芯材料等关键因素的影响，以优化磁性元件的性能。首先，我们将提取反映磁通密度分布特征及波形形状特征的变量，构建分类模型以识别励磁波形，并基于分类结果评估模型的合理性和有效性。其次，修正斯坦麦茨方程，构建适应温度变化的磁芯损耗预测模型，以提升其在实际应用中的预测精度。然后，结合实验数据，将探讨温度、励磁波形和磁芯材料对磁芯损耗的独立或协同影响，确定最优的低损耗工作条件。基于此，提出适用于多种磁性材料和工况条件的磁芯损耗预测模型，评估其精度与泛化能力，并进一步预测不同工况下的磁芯损耗表现。最后，以磁芯损耗与传输磁能为核心评价指标，构建优化模型，探讨在特定条件下实现最小磁芯损耗与最大传输磁能的最佳设计方案。

1.2 本文拟解决的问题

(1) **磁通密度与励磁波形分类**：基于实验数据，分析磁通密度的分布特性及励磁波形的形态特征，提取能够反映磁通密度分布与波形形状的特征变量。基于所提取的特征变量构建分类模型，分析识别三类主要的励磁波形（正弦波、三角波、梯形波），并评估分类模型的合理性与有效性。

(2) **斯坦麦茨方程修正**：针对同种磁芯材料，在正弦励磁波形下分析温度变化对磁芯损耗预测的影响。基于现有斯坦麦茨方程，构建能够适应不同温度变化的修正模型。将温度作为新增变量引入修正模型，提升其在温度变化较大场景下的损耗预测精度。

(3) **磁芯损耗影响因素分析**：通过实验数据，分析温度、励磁波形以及磁芯材料对磁芯损耗的独立及协同作用。量化不同条件下各因素对磁芯损耗的贡献，确定实现损耗最小化的因素组合。

(4) **数据驱动的磁芯损耗预测模型构建**：构建具有广泛适用性和高精度的磁芯损耗预测模型。评估其预测精度和泛化能力，并探讨其在行业应用中的价值。通过测试样本验证模型的有效性。

(5) **磁芯损耗与传输磁能的优化**：基于前述损耗预测模型，结合磁芯损耗与磁能传输两个评价指标，建立优化模型。通过分析温度、频率、励磁波形、磁通密度峰值及磁芯材料等因素组合，寻找实现磁芯损耗最小化和磁能传输最大化的最佳因素。

二、模型假设与符号说明

2.1 模型的基本假设

根据磁芯材料损耗数据和文中所给条件，本文做出如下假设：

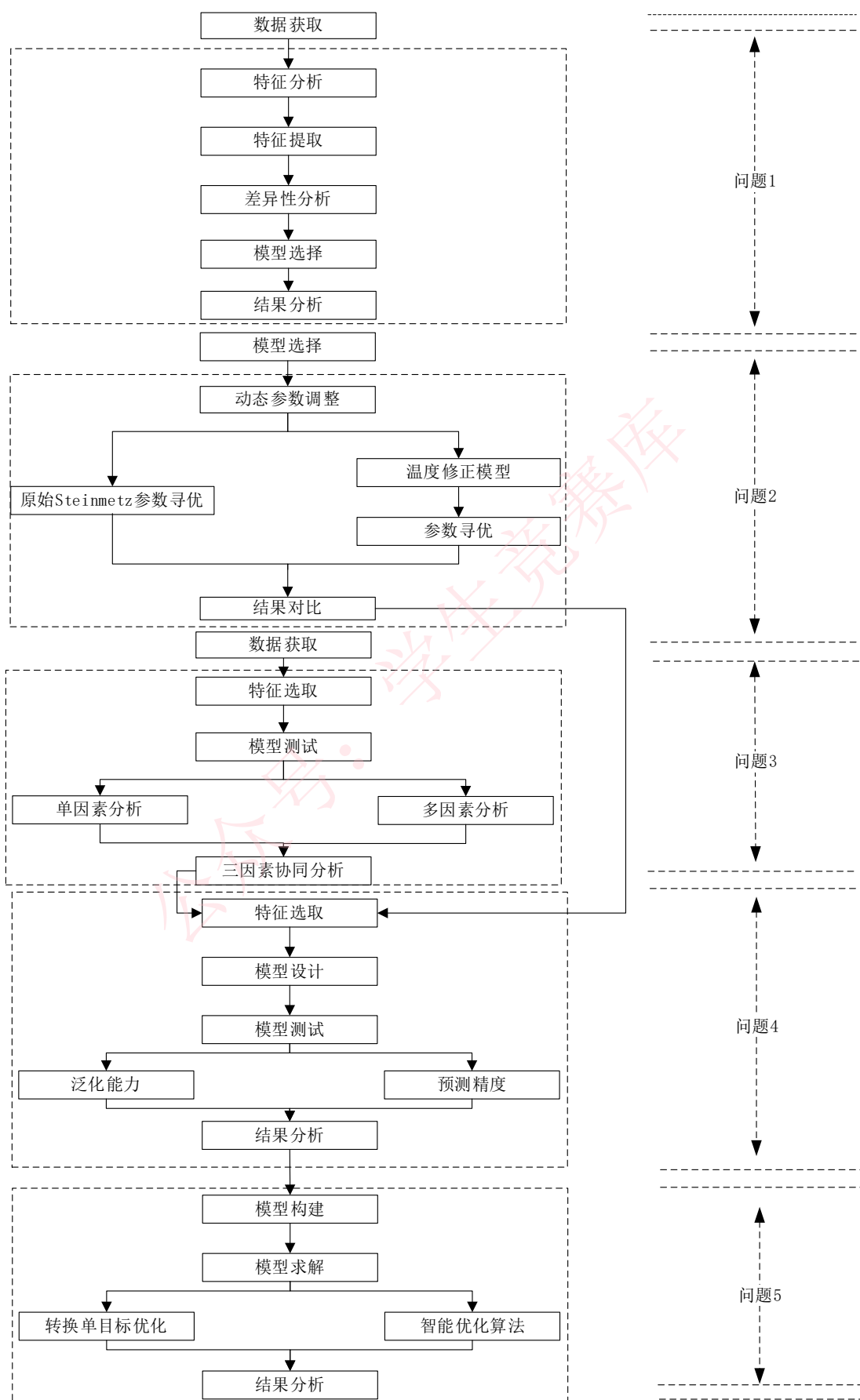
- (1) 假设磁芯材料数据真实有效。
- (2) 假设磁芯密度数据与磁芯损耗具有一定的相关性。
- (3) 假设磁芯损耗由附件所给数据表示，不考虑除附件给出数据外其他因素。

2.2 模型符号说明

表 2.1 符号说明

序号	符号	说明
1	R^2	决定系数
2	RMSE	均方根误差
3	MAPE	平均绝对百分比误差
4	P	磁芯损耗
5	f	频率
6	T	温度
7	B_m	磁通密度峰值
8	Recall	召回率
9	K-S	K-S检验
10	LSTM	长短时记忆网络

三、技术路线图



四、问题一的模型的建立与求解

4.1 问题分析

励磁波形作为影响磁芯性能的核心要素之一，其形态深刻影响着磁芯的损耗特性，其主要体现在磁通密度随时间变化的分布规律上^[5,6]，不同的励磁波形会导致磁通密度呈现出不同的增长、衰减或波动模式，因此需要对励磁波形进行准确识别。

本文使用磁通密度数据来对励磁波形进行分类。首先对题目提供的数据进行预处理。其次，绘制经过预处理后的磁通密度数据的频率直方图和波形图，通过图形分析磁通密度数据的分布以及不同波形的特征，提取磁通密度的时域特征变量；对经过预处理后的磁通密度数据做快速傅里叶变换，提取磁通密度的频域特征变量；对经过预处理后的磁通密度数据进行主成分分析，提取反映磁通密度数据特征的主成分变量。然后，在训练集上对提取的特征进行差异性分析，来获得最具分辨性的磁通密度波形特征。最后，使用支持向量机、决策树、集成学习算法实现不同励磁波形的分类。

4.2 模型建立与求解

4.2.1 数据分析与特征提取

(一) 数据分析

磁通密度数据是在各种工况下对磁芯材料进行实验测量得到的。在每个励磁波周期内共采样 1024 个点，即每种磁芯材料在特定工况下测量 1024 个磁通密度数据点。磁通密度数据能够反映施加在磁芯材料上的励磁波形特性。不同励磁波形（正弦波、三角波、梯形波）作用下，磁通密度呈现出不同的增长、衰减或波动模式，为识别励磁波形类型提供了重要的信息基础。正弦波、三角波和梯形波作用下的磁通密度数据可视化如图 4.1(a)、4.1(b)、4.1(c)所示。

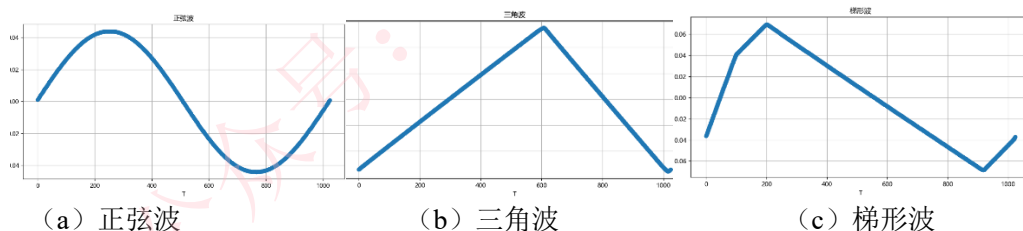


图 4.1 磁通密度可视化图

本任务旨在通过磁通密度数据识别施加在磁芯材料上的励磁波类型。由于数据量较大，且原始数据中可能存在缺失值和跳变数据等异常情况，对后续模型的分类结果产生不利影响。因此，首先需对磁通密度数据进行预处理。对于缺失数据，基于对磁通密度数据的可视化分析，可以观察到其数据表现出良好的平滑曲线特性。同时考虑到原始数据中包含非线性的正弦波形，本文选择 K 近邻(KNN)方法来填补缺失值。首先，通过滑动窗口异常检测方法识别异常数据点并将其剔除，然后使用 K 近邻方法对剔除的异常数据进行补充。这种预处理方法确保了数据的完整性与连续性，从而为后续模型分类提供了更为可靠的数据基础。

磁通密度数据在一个励磁波周期内等间距采样，以采样点为横坐标，磁通密度为纵坐标获取磁通密度波形图。为了区分不同磁芯材料在不同励磁波作用下的磁通密度波形图差异，帮助后续提取反映磁通密度分布及波形形状的特征分量提供初步分析依据。将每种磁芯材料磁通密度数据按照正弦波、三角波和梯形波将原始磁通密度数据分为 12 组，分别绘制 12 组磁通密度波形图，结果如图 4.2 所示。对所有磁通密度波形的可视化结果可以得知，所有的正弦波、三角波以及梯

形波均具有各自明显形状特点，正弦波曲线为丝滑曲线，三角波曲线存在两个拐点，梯形波存在四个拐点，材料 1 和材料 4 的采样取值周期起始点较为规律，波形分布较为规律，材料 2 和材料 3 的采样取值周期起始点较为混乱，波形分布较为混乱。

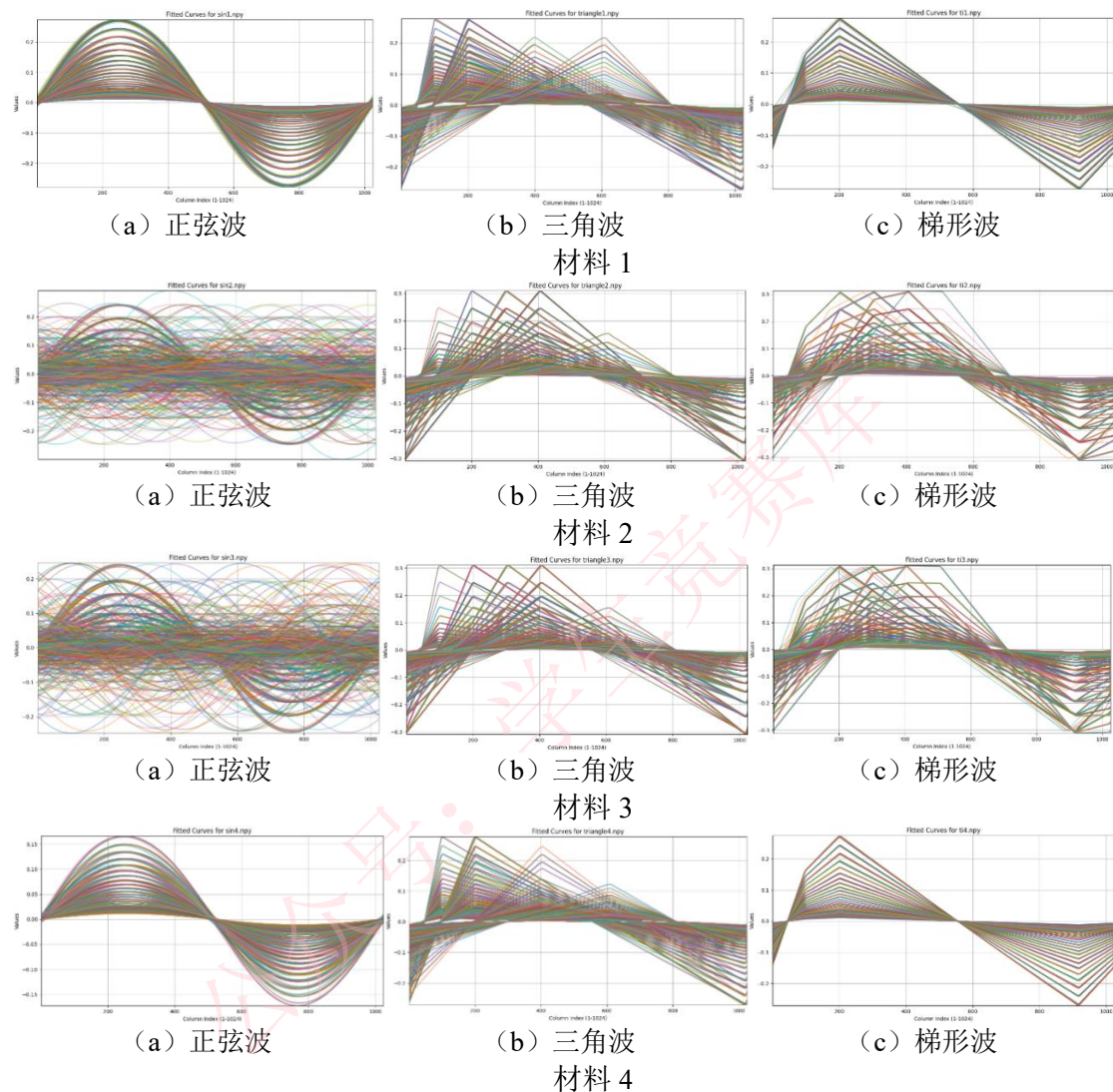


图 4.2 训练集中的磁通密度图

为了获取不同磁芯材料在不同励磁波作用下磁通密度的分布，基于上述数据分组，分别绘制 12 组磁通密度频率直方图，如图 4.3 所示。

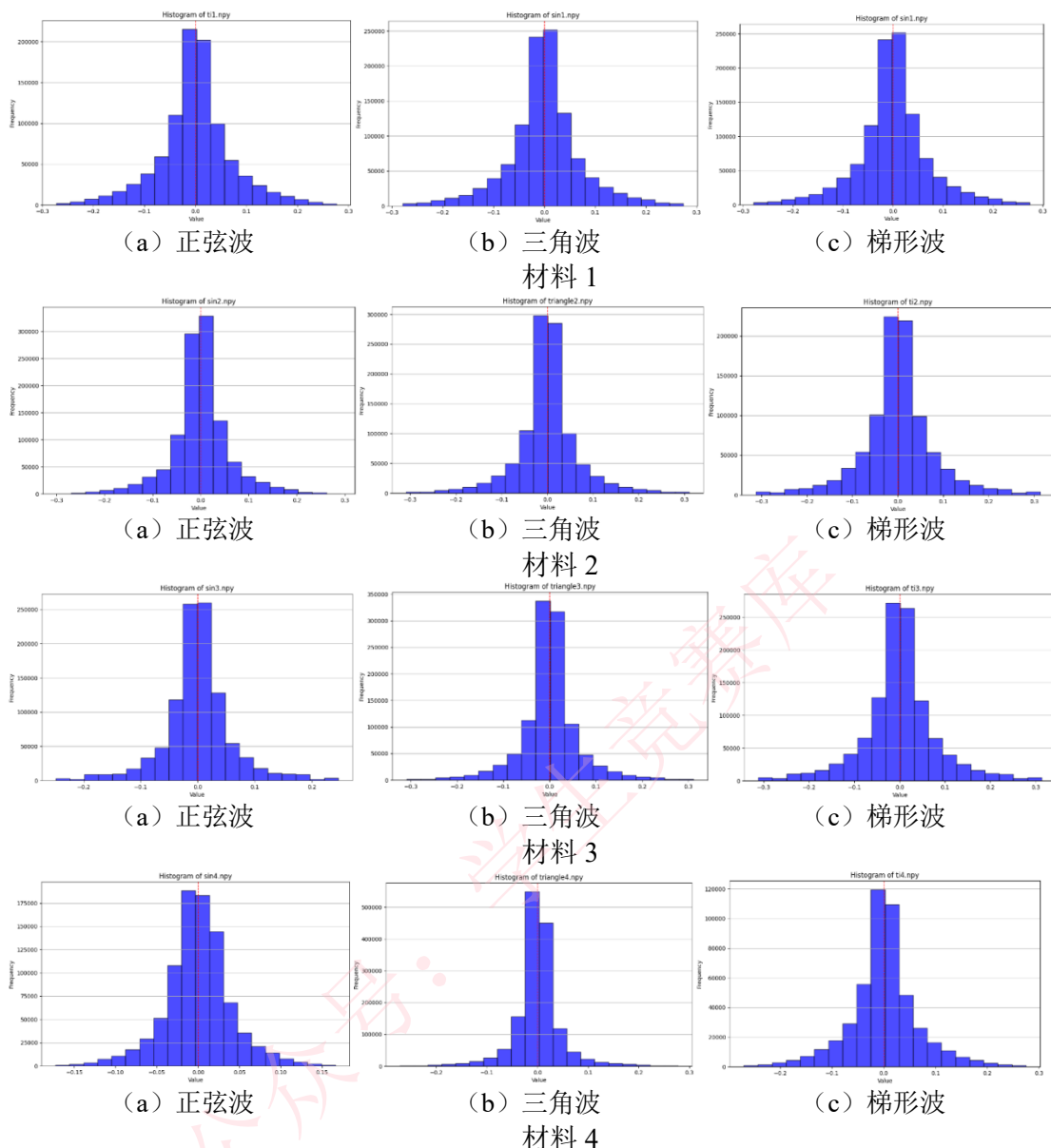


图 4.3 训练集中磁通密度频率直方图

(二) 特征提取

特征提取使用了时域特征、频域特征,另外使用主成分分析进行了特征提取。

(1) 时域特征

常用的信号时域特征有峰值、均方根误差、平均值、峰值因子、峰度、偏度。通过对所有磁通密度波形图可视化角度分析,不同材料的正弦波、三角波和梯形波的形状无明显区别特征,材料 1 和材料 4 的采样取值周期起始点较为规律,波形分布较为规律,材料 2 和材料 3 的采样取值周期起始点较为混乱,波形分布较为混乱。因此初步提取信号平均值、峰值、均方根误差、峰值因子。

通过对所有磁通密度频率直方图的可视化结果可知,不同波形的磁通密度频率分布呈现一定分布特征,左右数据分布存在一定差异。首先对不同组的磁通密度数据做 Kolmogorov-Smirnov 正态检验,通过检验结果可知,所有波形的磁通密度分布均不符合正态检验,因此初步信号提取峰度、偏度。

峰值因子(peak-to-average ratio,PAR)是描述信号峰值与有效值的比值的指标。

它表示了信号的峰值相对于有效值的倍数，能够反映信号的波形形状和脉冲特征。

均方根误差是预测值与真实值偏差的平方与观测次数比值的平方根。

峰度（Kurtosis）是统计学中衡量数据分布情况的另一特征指标，与偏度不同之处在于，峰度是从垂直维度上表征数据分布的峰态，即相较于正态分布而言当前数据峰态更加陡峭还是更加平滑。对于具有 n 个值的样本变量 X ，其峰度为样本的四阶标准中心矩。峰度的具体计算公式如式(4.1)所示。

$$Kurtosis = \frac{\frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})^4}{\left(\frac{1}{n} \sum_{i=1}^n (x_i - \bar{x})^2 \right)^2} - 3 \quad (4.1)$$

式中 x_i 表示样本 X 中的第 i 个值， $\bar{x}$ 表示样本 X 的均值。

按照峰态系数原始定义正态分布的峰度值为 3，但为了使正态分布的峰度值为 0，即，使峰度值的衡量以 0 作为基准，因此在实际应用中通常要对峰度值进行减 3 处理。即当峰度值 > 0 时，此时数据分布的峰态相较正态分布更加陡峭；当峰度值 < 0 时，此时数据分布的峰态相较正态分布更加平滑。峰度的示意图如图 4.4 所示，其中第一个子图就是峰度为 0 的情况，第二个子图是峰度大于 0 的情况，第三个则是峰度小于 0。

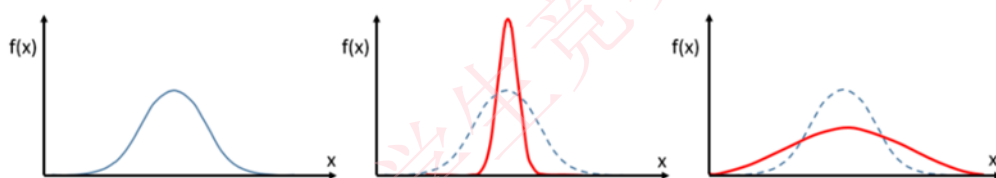


图 4.4 峰度示意图

偏度（Skewness）是表示非对称性的指标，用来衡量与正态分布相比左右偏斜的程度。样本偏度反映了样本数据与对称性的偏离程度和偏离方向，偏度的定义如式(4.2)所示。

$$Skew(X) = E\left[\left(\frac{X - \mu}{\sigma}\right)^3\right] = \frac{k_3}{\sigma^3} = \frac{k_3}{k_2^{3/2}} \quad (4.2)$$

式中 k_2 为二阶中心矩； k_3 为三阶中心距。

数据的偏离方向与偏离程度是相较于正态分布而言。如果数据分布与正态分布一样呈对称分布，则偏度为 0。当偏度 > 0 时，数据分布相较于正态分布整体呈现偏向右侧的情况，此时称之为右偏或正偏；当偏度 < 0 时，数据分布相较于正态分布整体呈现偏向左侧的情况，此时称之为左偏或负偏。同时偏度绝对值的大小表征着数据偏离程度的高低。正负偏的分布如图 4.5 所示，在图中，左边的是正偏，右边的是负偏。

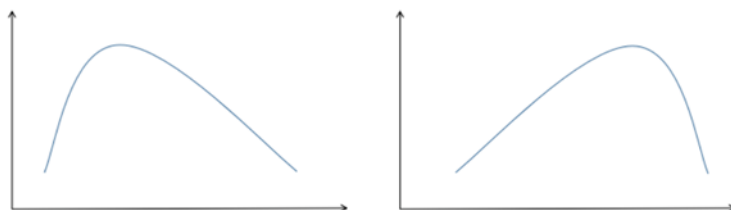


图 4.5 正负偏分布情况图

(2) 频域特征

针对信号的频域特征,对附件一中不同波形对应的磁通密度数据做快速傅里叶变换,通过结果分析可知,所有波形峰值频率相同,不同信号对应带宽和谐波比存在一定差异,因此初步提取信号带宽、谐波比。

信号带宽(signal bandwidth)是用于衡量信号所覆盖频率范围的一个重要参数。带宽定义为信号包含的最高频率与最低频率之差,即信号的频率范围。它通过分析信号的频谱图来确定信号中所包含的各频率成分,反映信号的频率分布情况。带宽越宽,意味着信号的频率变化范围越大,涵盖更多的谐波成分。

(3) 主成分分析

主成分分析(Principal Component Analysis, PCA)是一种多变量统计方法,通过正交变换将 n 维特征映射到 m 维上($n>m$),实现数据降维,映射后的 m 维特征被称为主成分。主成分分析通过协方差矩阵的特征值分解,提取数据中的主要特征,实现了对高维数据的有效降维和特征提取。

进一步,采用 PCA 提取区分不同波形的主成分特征。通过所有磁通密度波形的可视化结果可以看出,采样周期的起始点不同导致了正弦波的起始位置存在较大差异,会导致主成分分析时无法有效提取不同波形之间的主成分因子特征。因此首先对所有波形的磁通密度数据进行一定平移,使得不同波形的起始位置相同,在提取不同波形主成分因子过程中,将对齐后的磁通密度数据作为输入。通过图 4.6 发现,前 5 个主成分的累计解释方差比达到了 99%,所以从 1024 个磁通密度波形特征输入中,提取出了 5 个主成分。

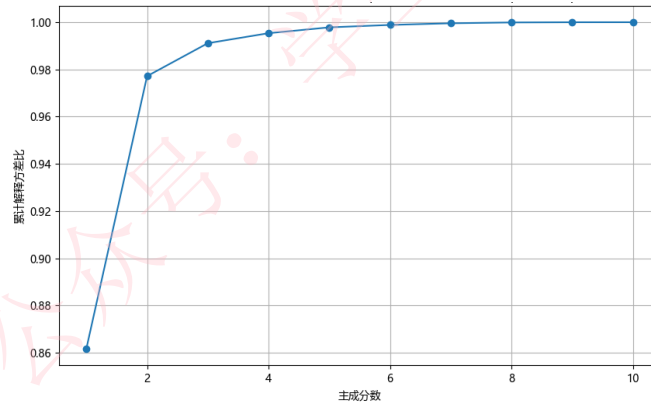


图 4.6 前十个主成分的累计解释方差比

最终,结合上述提取的所有特征变量,共初步提取信号平均值、峰值、均方根误差、峰值因子、峰度、偏度、带宽、谐波比、主成分 1、主成分 2、主成分 3、主成分 4、主成分 5 共 13 个特征变量。

通过对磁通密度数据检验可知,不同波形的磁通密度分布不符合正态分布,因此采用克鲁斯卡尔-沃利斯检验(Kruskal-Wallis, K-W)方法和曼-惠特尼 U 检验(Mann-Whitney U test)对不同波形间的特征差异进行分析。

克鲁斯卡尔-沃利斯检验是一种用于比较两个或两个以上独立样本是否来自相同分布的非参数统计方法。该检验不要求样本数据满足正态分布假设,因而适用于不满足正态性或方差齐性条件的数据。K-W 检验通过对样本数据进行排序并分析其秩次来评估各组间的差异,广泛用于处理小样本、偏态分布或离群值较多的情况。

曼-惠特尼 U 检验,是一种用于比较两组独立样本是否来自相同分布的非参数统计方法。该检验不要求数据服从正态分布,因而适用于非正态分布或方差不

齐的情况。通过比较两组样本的秩次，曼-惠特尼 U 检验评估组间的差异，特别适合处理小样本或含有异常值的数据。

首先采用 Kruskal-Wallis 检验方法对上述特征变量进行三种波形的分类检验方法，具体 p 值如表 4.1 所示，其中，当 p 值为 0 时说明特征变量对三种波形的区分效果好。进一步采用 Mann-Whitney U 检验方法对上述三分类效果好的特征变量进行两两波形分类检验，结果如表 4.1 所示，检验数值为 0 说明特征变量对两两波形分类效果好，最终选取对两两波形之间区分效果好的特征变量，从表中数据可知，峰值因子、峰度可以很好区分三种波形，最终选取特征变量峰值因子、峰度用于区分不同波形。

表 4.1 不同波形之间的特征差异分析

特征	p 值	正弦波 vs 三角波	正弦波 vs 梯形波	三角波 vs 梯形波
均值	0.22	0.83	0.83	0.52
峰度	0	0	0	0
最大值	1.23e-76	3.18e-14	1.18e-87	3.70e-41
峰值因子	0	0	0	0
偏度	0	0	0	2.37e-66
标准差	2.55e-80	0.000	5.87e-44	8.49e-68
带宽	0	0	0	4.70e-11
谐波比	0	0	0	1.43e-5
主成分因子 1	8.18e-82	2.69e-20	3.81e-20	4.75e-72
主成分因子 2	0	0	0	7.89e-21
主成分因子 3	0	1.12e-116	0	2.54e-168
主成分因子 4	2.4e-35	0.003	3.95e-17	3.01e-67
主成分因子 5	0	1.51e-67	0	9.30e-44

4.2.2 支持向量机

支持向量机模型（Support Vector Machine, SVM）是目前最广泛使用的模型之一。支持向量机的基本原理是寻找一个分类超平面，该超平面不仅可以准确区分不同类型的数据，而且还可以最大限度的增大分割间隔。可以使分类结果鲁棒性最好，泛化能力最强。其核心思想是在特征空间中寻找支持向量，构造一个可以使不同类的间隔最大化的分类器。

求解支持向量机的决策边界就是求解一个凸二次规划。支持向量机使用 Hinge Loss 作为损失函数，这使得 SVM 更具鲁棒性，Hinge Loss 损失函数定义如式(4.3)所示。

$$l(f(x^n), \hat{y}^n) = \max(0, 1 - \hat{y}^n f(x^n)) \quad (4.3)$$

式中 $l(\bullet)$ ——损失函数；

$\hat{y}^n$ ——实际所属类别，取值为 ± 1 。

SVM 的实质是对线性可分的数据进行分析，但在实际应用中，需要分类的数据通常在其所在的特征平面内是线性不可分的，因此 SVM 使用 Kernel Trick 将低维空间非线性数据映射至高维特征空间使其线性可分。

分类过程可由图 4.7 描述。图中待分类的两种数据分别由“×”与“O”表示，两

类数据标签分别记为+1 与-1, w 与 b 是超平面参数。

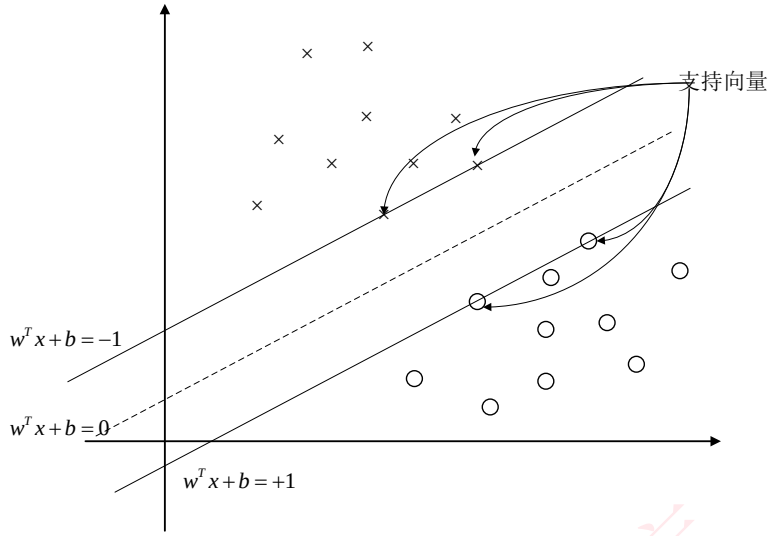


图 4.7 SVM 解决二分类问题

SVM 寻找最优决策边界的过程可由式(4.4)进行表述:

$$\underset{w,b}{\text{minimize}} \quad \frac{1}{2} w^T w + C \sum_{i=1}^N \max(0, 1 - y_i (w^T \Phi(x_i) + b)) \quad (4.4)$$

式中 w , b 代表超平面参数;

C 代表损失函数的惩罚系数;

y_i 代表第 i 个数据所属类别标签;

$\Phi(x_i)$ 代表核函数。

式(4.4)可以进一步转化成一种包含约束的二次规划求解问题, 具体描述如式(4.5)所示。

$$\begin{aligned} & \underset{w,b,\xi,\xi^*}{\text{minimize}} \quad \frac{1}{2} \|w\|^2 + C \sum (\xi_i + \xi_i^*) \\ & \text{subject to} \quad \begin{cases} y_i - \langle w, \Phi(x_i) \rangle - b \leq \varepsilon + \xi_i \\ \langle w, \Phi(x_i) \rangle + b - y_i \leq \varepsilon + \xi_i^* \\ \xi_i, \xi_i^* \geq 0 \end{cases} \end{aligned} \quad (4.5)$$

式中 C 代表损失函数的惩罚系数;

$\langle w, x_i \rangle$ — 数据点积;

ξ_i, ξ_i^* — 松弛变量; Φ — 核函数。

问题(3.5)可以使用对称 Lagrange 法解算:

$$\begin{cases} \underset{a_i}{\text{minimize}} & -\frac{1}{2} \sum_{i=1}^N \sum_{j=1}^N y_i y_j a_i a_j \Phi(x_i) \Phi(x_j) + \sum_{i=1}^N a_i \\ \text{subject to} & \sum_{i=1}^N y_i a_i = 0, 0 \leq a_i \leq C, \forall i \end{cases} \quad (4.6)$$

式中 a_i 代表 Lagrange 因子;

N 代表输入特征数量。

设 a^* 是对偶问题最优解算, $H(w^*, b^*)$ 表示超平面, 定义如式(4.7)所示。

$$\begin{cases} I_s := \{i: a_i^* > 0\} \\ S := \{x_i: i \in I_s\} \end{cases} \quad (4.7)$$

式中 S 表示为支持向量的集合。

计算出的分类超平面参数 w^* 与 b^* ，表达式如式(4.8)所示。

$$\begin{cases} w^* = \sum_{i \in I_s} a_i^* y_i x_i \\ b^* = y_i - \sum_{i \in I_s} a_i^* y_i \Phi(x_i) \Phi(x) \end{cases} \quad (4.8)$$

4.2.3 决策树

决策树模型（Decision Tree）是一种基于树结构进行决策分类的常见的机器学习算法。决策树分类过程中每次选择一个特征作为分叉，因此在每一个节点处选择何特征来分叉以达到更好的分类效果是关键问题。学习决策树通常包括三个步骤：选择特征、创建决策树和修剪决策树。

决策树通常由多个节点构成，其中包含一个根节点与多个叶子节点。根节点是初始分裂节点，包含一组完整的样本，每个叶子节点对应一条结论即表示一个分类类别，而其他内部节点表示一个个判断条件，从根节点到叶子节点的每条路径分别对应一个测试样本。当使用决策树分类时，从根节点开始，每个节点根据样本特征进行测试，依据测试结果对其包含的样本集进行划分，划分后的子集分配到不同的子结点中，重复上述分配过程直至达到叶子节点，完成样本的分配。决策树学习的目的是创建一个具有较强泛化能力的模型，使其具备处理未知样本的能力。决策树模型结构示意图如图 4.8 所示，图中矩形节点代表叶子节点。

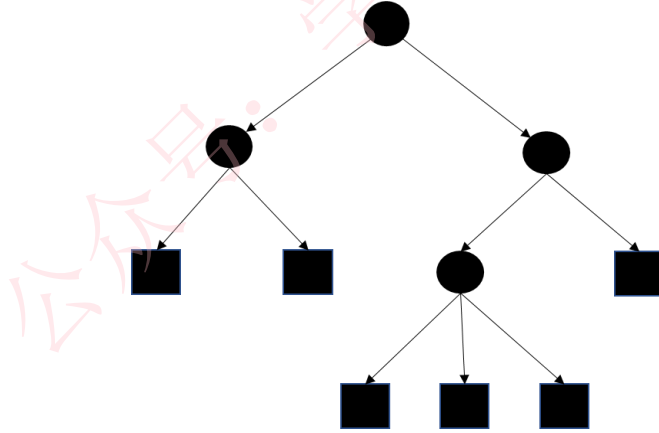


图 4.8 决策树模型

假设输入具有 m 个样本，每个样本含 n 个离散特征，记特征集合为 A ，样本输出集合为 D ，信息增益的阈值为 ε ，决策树模型的基本流程如图 4.9 所示。

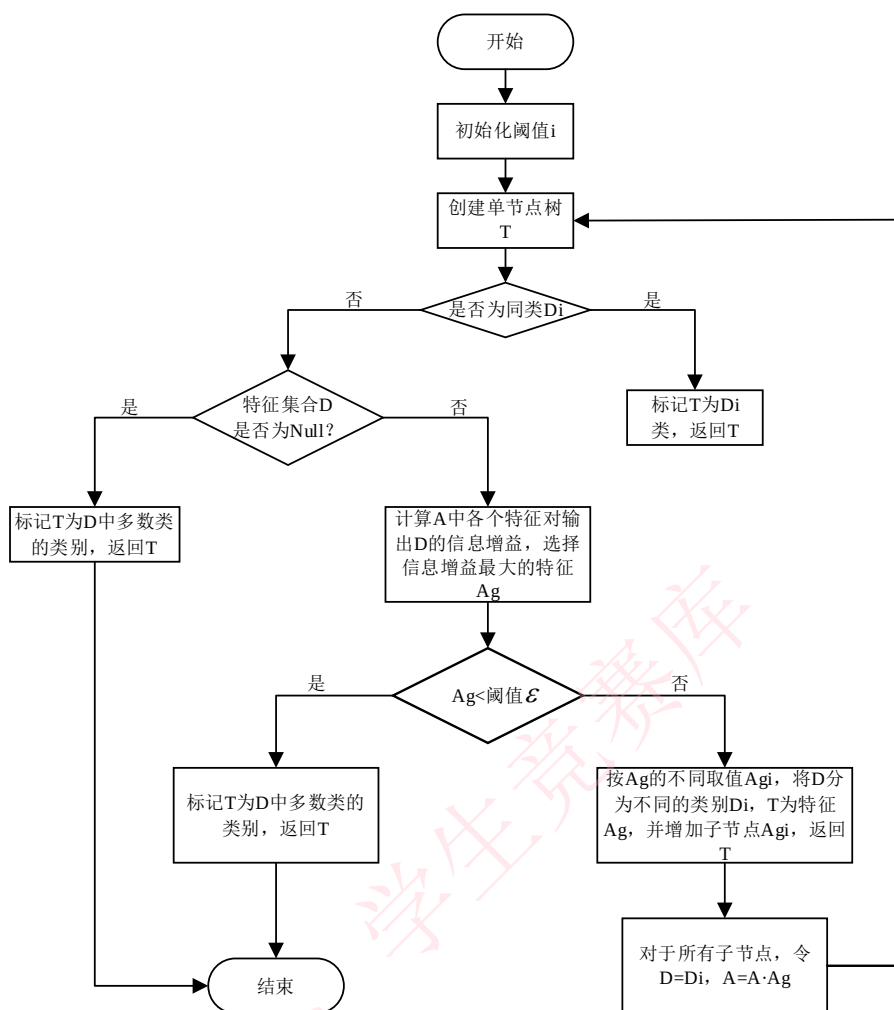


图 4.9 决策树算法流程图

4.2.4 集成学习

集成学习方法通过构建并结合多个学习器来解决单个预测问题, 也称为多分类系统。每个学习器独立学习并进行预测。各个预测结果最终被合并为一个预测结果, 使其比任何分类模型单独进行预测的结果更好。集成学习方法通常假设多个模型的组合可以产生更强大的模型。使用集成学习模型分类过程中, 即使某个单独分类器得到错误的预测, 其他分类器可以纠正错误, 最终可以得到正确的预测结果。集成学习模型原理如图 4.10 所示。

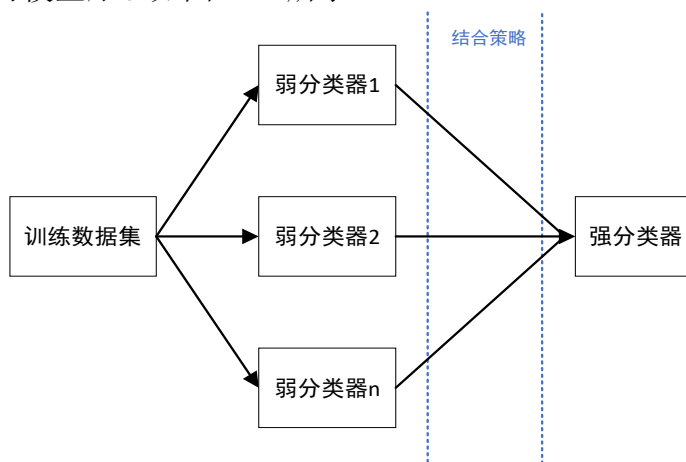


图 4.10 集成学习模型

本研究使用了 Stacking 集成学习，Stacking 集成学习属于异质性集成，由不同的基学习器搭建集成模型。Stacking 算法模型如图 4.11 所示。

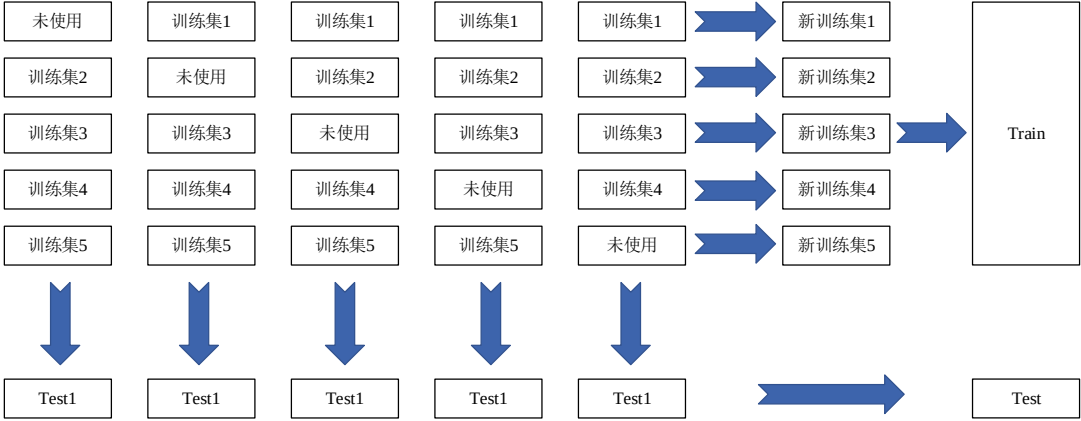


图 4.11 Stacking 算法模型图

4.2.5 模型分析

K 折交叉验证是机器学习中用于模型性能评估与选择的一种流行技术，其基本思想是将数据集拆分为 K 个子集，然后进行 K 次独立的训练和评估。在每次训练过程中，依次选取 K 个子集中的一个用作验证集，而剩余的 K-1 个子集用于训练，然后通过 K 次训练的结果进行平均来计算模型的最终性能。

由于 K 折交叉验证不是简单地在单个数据分割上进行训练和评估，而是基于多次迭代可以使我们能够最有效地利用可用数据。特别是在数据集大小有限的情况下，通过对多次训练的结果进行平均，有助于减少数据中随机变化对模型评估的影响、减少性能评估的偏差和方差，提供对模型性能的更可靠的估计。使我们能够较为准确的评估模型的泛化性能，即模型在未知数据上的表现。

本文采用 K 为 5 的 5 折交叉验证选取最优超参数，有效避免模型过拟合。

分类模型常用的评价指标包括准确率、精度、召回率、F1 分数、混淆矩阵、ROC 曲线、AUC 分数和均方误差，本文主要从准确率、召回率、F1 分数三个方面评估模型分类结果，以混淆矩阵的形式对分类结果进行展示。其中准确率是机器学习中常用的评估指标，用于衡量模型做出正确预测的比例，它被定义为模型做出的正确预测的数量除以预测的总数。准确率是一种简单直观的指标，易于理解和解释。然而，当数据集不平衡或误报和漏报的成本不同时，它可能并不总是一个合适的指标。而理解召回率与 F1 分数，True Positives(TP)，即正确分类的正样本；False Positives(FP)，即错误分类的正样本；True Negatives(TN)是错误分类的负样本，False Negatives(FN)是错误分类的负样本，直观展示如表 4.2 所示。

表 4.2 TP、FP、TN、FN 示意表

真实标签 \ 预测标签	正例	反例
正例	TP（真正类）	FN（假反类）
反例	FP（假正类）	TN（真反类）

召回率衡量正确分类的正样本占有所有实际正样本的比例其定义式如式(4.9)所示。召回率的取值范围为[0,1]，具有高召回率的模型意味着它可以正确识别高

比例的实际阳性样本，同时最大限度地减少假阴性的数量。

$$R = \frac{TP}{TP + FN} \quad (4.9)$$

F1 分数是将精度和召回率相结合的一个指标，其取值范围为[0,1]。当我们存在数据不平衡的情况或当我们想要平衡模型中精确度和召回率的重要性时，它特别有用，其定义式如式(4.10)所示。

$$F_1 = \frac{2 * P * R}{P + R} \quad (4.10)$$

式中 R 表示召回率，P 表示精度，其衡量模型分类为正的所有样本中正确分类的正样本所占的比例，其定义如式(4.11)所示。

$$P = \frac{TP}{TP + FP} \quad (4.11)$$

根据前面介绍的提取选择出的峰度和峰值系数两种波形特征以及机器学习分类模型理论，对支持向量机、决策树和集成学习三种模型进行实验，研究不同模型对励磁波形的分类识别结果。

在将决策树、支持向量机和集成学习（决策树+支持向量机）3 个模型搭建完成后，并在完整的训练集上完成最终训练，在测试集依次评估模型效果，求出最终生成模型的平均 Accuracy、平均 Recall 和平均 F1-score 值，并利用热力图将混淆矩阵进行展示。三个机器学习算法模型比较的分类情况如表 4.3 所示。

表 4.3 模型分类情况

模型类型	准确率	召回率	F1 分数
决策树	1.00	1.00	1.00
支持向量机	1.00	1.00	1.00
集成学习	1.00	1.00	1.00

由表 4.3 可以看出，在本章选取的峰度和峰值因子两种磁通密度波形特征的基础上，三种模型分类准确率均高达 100%，这可能与数据集质量高，特征选择的有效有关，进一步证明了所提波形特征在分类励磁波形方面的有效性。

假定 1 为正弦波、2 为三角波、3 为梯形波。基于上述分类模型，首先求解出附件 2 中训练集数据的峰值因子和峰度作为输入，获取最终附件 2 中所有训练集的分类结果。最终附件 2 中三种波形各自数量分类结果如表 4.4 所示，样本 1、5、15、25、35、45、55、65、75、80 分类结果如表 4.5 所示。

表 4.4 附件 2 中三种波形的各自数量

励磁波类型	正弦波	三角波	梯形波
数量	20	44	16

表 4.5 附件 2 中 1、5、15、25、35、45、55、65、75、80 分类结果

样本序号	分类结果
1	2（三角波）
5	2（三角波）
15	1（正弦波）
25	2（三角波）
35	3（梯形波）
45	3（梯形波）
55	2（三角波）
65	2（三角波）
75	2（三角波）
80	1（正弦波）

4.3 本章小结

本节首先对提供的数据进行了系统的预处理，以确保数据的完整性和可靠性。随后，通过绘制频率直方图和波形图，对经过预处理的磁通密度数据进行可视化分析，揭示了其分布特征及不同波形的差异，从中提取出相关的时域特征变量。接着，采用快速傅里叶变换对磁通密度数据进行频域分析，提取出频域特征变量。

此外，通过主成分分析，提取出反映磁通密度数据特征的主成分变量。在训练集上进行差异性分析识别最具分辨性的磁通密度波形特征。最后，本章利用支持向量机、决策树和集成学习算法对不同励磁波形进行了分类实验，展示了不同分类方法的有效性和优越性。

五、问题二的模型的建立与求解

5.1 问题分析

在传统的磁芯损耗模型中，斯坦麦茨方程（Steinmetz equation）虽然作为经典模型被广泛应用，但其适用性受到限制。该方程主要针对正弦波形设计，且对于不同类型的磁芯材料及工作温度的变化，斯坦麦茨方程可能会导致较大的误差。这在实际工程应用中带来了很多不便和复杂性。目前已经有一些方法对非正弦波形下磁芯损耗模型进行修正，但对温度因素还没有一个较好的模型能够适应不同温度变化。

针对材料 1 的正弦波形的温度变化因素，首先，本文使用粒子群算法、模拟退火算法和最小二乘法对原始斯坦麦茨方程常数参数 k_1 、 α_1 、 β_1 进行寻优，并对粒子群算法进行了改进，以提高原始斯坦麦茨方程的准确率；然后，本文构造了几种可适用与不同温度变化的磁芯损耗修正方程，包括线性、二次、三次、指数、幂级数温度修正方法。最后，本文对构造的温度修正方程进行了四种方法的参数寻优，结果表明加入温度因素的修正方程比原始方程预测磁芯损耗效果更佳。

5.2 模型建立与求解

5.2.1 最小二乘法

最小二乘法是解决回归分析、数据拟合的常用技术，旨在通过最小化误差的平方和从而寻找拟合数据的最佳函数。对于给定的一组数据点 (x_i, y_i) 满足理论函数 $y = f(w, x)$ ，最小二乘法旨在寻找一个目标函数，即获取合适的函数参数 w 使得函数预测值 $f(w, x_i)$ 与实际值 y_i 之间的误差平方和最小化，最终获得拟合数据的最佳函数。

误差平方和定义如式(5.1)所示：

$$SSE = (y_i - f(w, x_i))^2 \quad (5.1)$$

式中， y_i 为实际值， $f(w, x_i)$ 为函数预测值。

5.2.2 模拟退火算法

模拟退火算法是一种用于求解优化问题的随机算法，思想来源于固体退火原理。它用于寻找全局最优解，尤其适合于复杂的组合优化问题^[9]。模拟退火算法包含两个部分，即 Metropolis 算法和退火过程，分别对应内循环和外循环。外循环即退火过程，将固体达到较高的温度，然后按照降温系数使温度按照一定比例下降，当达到终止温度时，冷却结束，即退火过程结束。内循环即 Metropolis 算法，其设计有接受概率，在寻找到最优解后不会立即结束，而是通过接受概率与一个 0~1 之间的随机数进行比较，判断是否继续移动，实现了全局最优解的求解。

算法接受新解概率 P 的计算公式为：

$$p = e^{-\frac{\Delta f}{T}} = e^{-\frac{f(n') - f(n)}{T}} \quad (5.2)$$

式中， T 为当前温， Δf 为新解 $f(n')$ 与旧解 $f(n)$ 的差值。

模拟退火算法寻找最优解示意图如图 5.1 所示。

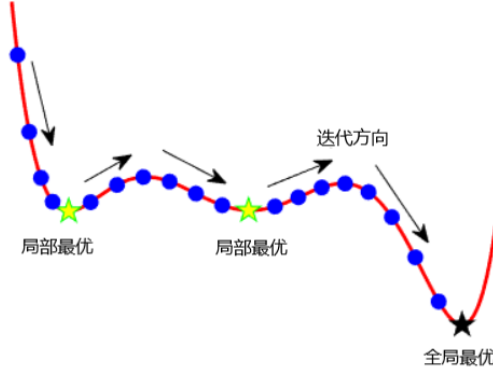


图 5.1 模拟退火算法寻找最优解示意图

5.2.3 加入动态参数调整的粒子群算法

粒子群算法是一种进化计算技术,最早由 Russel Eberhart 和 James Kenedy 提出,思想来源于鸟群的觅食行为,旨在通过群体中个体之间的协作和信息共享来寻找最优解。粒子群算法的基本思想是通过设计一种无质量的粒子来模拟鸟群中的鸟,粒子有两种属性:速度和位置,速度代表移动的快慢,位置代表一定的方向。每个粒子在搜索空间中单独的搜寻最优解,并将其记为当前个体极值,并将个体极值与整个粒子群里的其他粒子共享,找到最优个体极值作为整个粒子群的当前全局最优解,粒子群中的所有粒子根据自己找到的当前个体极值和整个粒子群共享的当前全局最优解来调整自己的速度和位置。算法按下列公式对每个粒子的位置信息和速度信息进行更新,位置更新公式如式(5.3)所示:

$$x_i^{(t+1)} = x_i^{(t)} + v_i^{(t+1)} \quad (5.3)$$

其中, $x_i^{(t+1)}$ 是粒子 i 在下次迭代 $t+1$ 的位置, $x_i^{(t)}$ 和 $v_i^{(t+1)}$ 分别是粒子 i 在当前迭代的位置和下次迭代的速度。

速度更新公式:

$$v_i^{(t+1)} = w \cdot v_i^{(t)} + c_1 \cdot r_1 \cdot (pbest_i - x_i^{(t)}) + c_2 \cdot r_2 \cdot (gbest - x_i^{(t)}) \quad (5.4)$$

其中, $v_i^{(t+1)}$ 是粒子 i 在下次迭代 $t+1$ 的速度; w 是惯性权重,控制粒子速度的保留程度; $v_i^{(t)}$ 是粒子 i 在当前迭代 t 的速度; c_1 和 c_2 是加速系数,分别代表个体学习因子和社会学习因子控制粒子向个体最优和全局最优靠拢的程度; r_1 和 r_2 是 $[0,1]$ 区间的随机数,增加随机性; $pbest$ 是粒子 i 迄今为止找到的个体最优位置; $gbest$ 是整个粒子群迄今为止找到的全局最优位置; $x_i^{(t)}$ 是粒子 i 在当前迭代的位置。

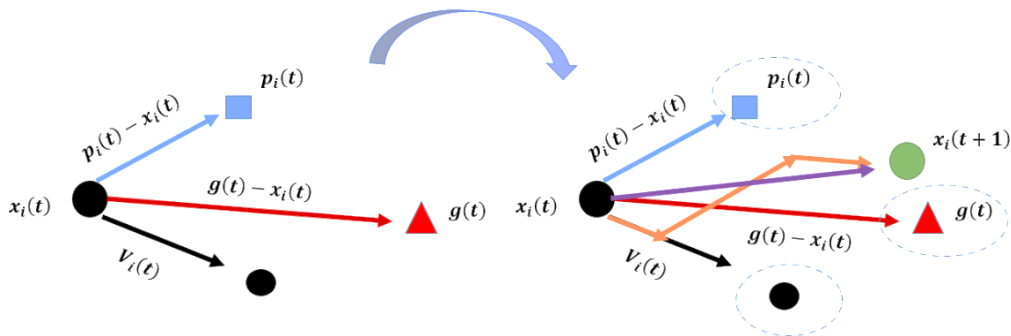


图 5.2 粒子迭代更新示意图

粒子群算法中的参数（惯性权重和学习因子）通常是固定的，可能导致收敛速度较慢。加入参数动态调整可以帮助算法在不同阶段自适应地调整搜索策略。因此，本文在粒子群算法中接入了惯性权重和学习因子的参数自适应调整策略。惯性自适应调整策略如式(5.5)所示。

$$w = 0.9 - \left(0.5 \cdot \frac{\text{iter}_{\text{num}}}{\text{max}_{\text{iter}}} \right) \quad (5.5)$$

式中， w 为惯性权重， iter_{num} 为当前迭代次数， max_{iter} 为最大迭代次数。

认知成分为粒子对自身历史最佳位置的影响因子，本文采用的自适应调整策略如式(5.6)所示。

$$c_1 = 0.5 + \left(1.0 \cdot \frac{\text{iter}_{\text{num}}}{\text{max}_{\text{iter}}} \right) \quad (5.6)$$

式中， c_1 为认知成分， iter_{num} 为当前迭代次数， max_{iter} 为最大迭代次数。

社会成分为粒子对全局最佳位置（群体最佳解）的影响因子，本文采用的自适应调整策略如式(5.7)所示。

$$c_2 = 0.5 + \left(1.0 \cdot \frac{\text{iter}_{\text{num}}}{\text{max}_{\text{iter}}} \right) \quad (5.7)$$

式中， c_2 为认知成分， iter_{num} 为当前迭代次数， max_{iter} 为最大迭代次数。

5.2.4 原始斯坦麦茨方程参数寻优

要探寻加入温度后的斯坦麦茨方程对预测磁芯损耗效果的影响，首先需要对原始的斯坦麦茨方程进行参数寻优，寻找其最好预测结果。

首先，对材料 1 中的正弦波励磁波数据进行提取，一共提取到 1067 个正弦波数据，其中 25°C 数据 271 个，50°C 数据 256 个，70°C 数据 265 个，90°C 数据 275 个。

然后，对斯坦麦茨方程参数进行寻优，斯坦麦茨方程如式(5.8)所示。

$$P = k_1 \cdot f^{\alpha_1} \cdot B_m^{\beta_1} \quad (5.8)$$

其中： P 为磁芯损耗； f 是频率； B_m 是磁通密度的峰值； k_1 、 α_1 、 β_1 是根据实验数据拟合的系数，一般 $1 < \alpha_1 < 3$ ， $2 < \beta_1 < 3$ ，但 k_1 、 α_1 、 β_1 与温度无关，首先对不同温度下的常数进行参数寻优，分析温度对参数的影响。本文使用最小二乘法对四种温度下的斯坦麦茨方程参数进行寻优。

四种温度下的最佳 k 、 α 、 β 值寻优结果如图 5.3 所示。从图中可以看出，温度升高时， k 值减小， α 、 β 都有所增大。因此，温度对斯坦麦茨方程常数参数有影响，需要对温度因素进行修正。

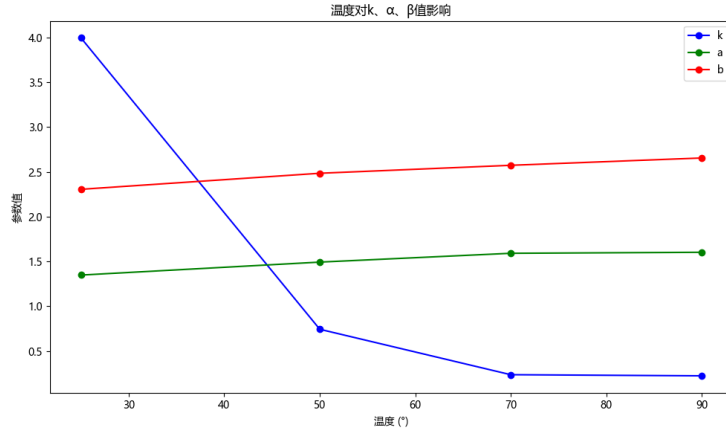


图 5.3 不同温度下 k 、 α 、 β 值对比

然后，本文对不同温度下以及所有温度下的斯坦麦茨方程结果进行了对比，评价指标采用决定系数 (Coefficient of Determination, R^2)、均方根误差 (Root Mean Square Error, RMSE) 和平均绝对百分比误差 (Mean Absolute Percentage Error, MAPE)。具体计算公式分别如式 (5.9)、式 (5.10) 和式 (5.11) 所示。评价结果见表 5.1。

$$R^2 = 1 - \frac{\sum (y_i - \hat{y}_i)^2}{\sum (y_i - \bar{y})^2} \quad (5.9)$$

其中， y_i 为实际值， $\hat{y}_i$ 为预测值， $\bar{y}$ 为实际值的平均值。

$$\text{RMSE} = \sqrt{\frac{1}{n} \sum (y_i - \hat{y}_i)^2} \quad (5.10)$$

其中， y_i 为实际值， $\hat{y}_i$ 为预测值， n 为样本数量。

$$\text{MAPE} = \frac{1}{n} \sum \left| \frac{y_i - \hat{y}_i}{y_i} \right| \times 100\% \quad (5.11)$$

其中， y_i 为实际值， $\hat{y}_i$ 为预测值， n 为样本数量。

表 5.1 不同温度对比下的评价指标

	25°	50°	70°	90°
R^2	0.9979	0.9984	0.9962	0.9936
RMSE	9615.3950	7052.6992	9299.7046	10854.8127
MAPE	17.5474	9.8620	14.9031	22.6502

从表 5.1 中可以看出，对单独温度进行参数寻优时， R^2 、RMSE 和 MAPE 评价指标都优于所有温度一起寻优时，这表明斯坦麦茨方程不能适应不同温度情况下的磁芯损耗值预测。

接着，本文采用四种方法（粒子群算法、参数动态调整的粒子群算法、模拟退火算法和最小二乘法）对所有温度下的斯坦麦茨方程参数进行优化。寻优预测结果见表 5.2。从表 5.2 可以看出，四种方法在寻找斯坦麦茨方程参数方面的效果相差不大，决定系数 R^2 均为 0.9448。这表明，在多温度条件下，斯坦麦茨方程的预测效果仍有待改善。

表 5.2 不同方法对比下的评价指标

	最小二乘法	模拟退火	粒子群	动态粒子群
R^2	0.9448	0.9448	0.9448	0.9448
RMSE	40474.5657	40474.5657	40474.5680	40474.5656
MAPE	35.6627	35.6634	35.6802	35.6626

参数寻优结果如表 5.3 所示。从表 5.3 可以看出，四种方法对三个参数的寻优结果差距不大，决定系数 R^2 都为 0.9448。这是由于数据量较多并且需要拟合的参数较为简单，且数据质量较好，无异常值等，导致四种方法都能较好的拟合到参数。

表 5.3 不同方法对参数寻优结果

	最小二乘法	模拟退火	粒子群	动态粒子群
k	1.4997	1.4997	1.50	1.4999
α	1.4296	1.4296	1.4296	1.4296
β	2.4713	2.4712	2.4712	2.4712

然后，本文对四种方法的预测结果进行了可视化展示，展示如图 5.4 所示。从四个图可以看出，四种方法的拟合平面都与真实数据差距不大，整体来看符合数据的变化情况。

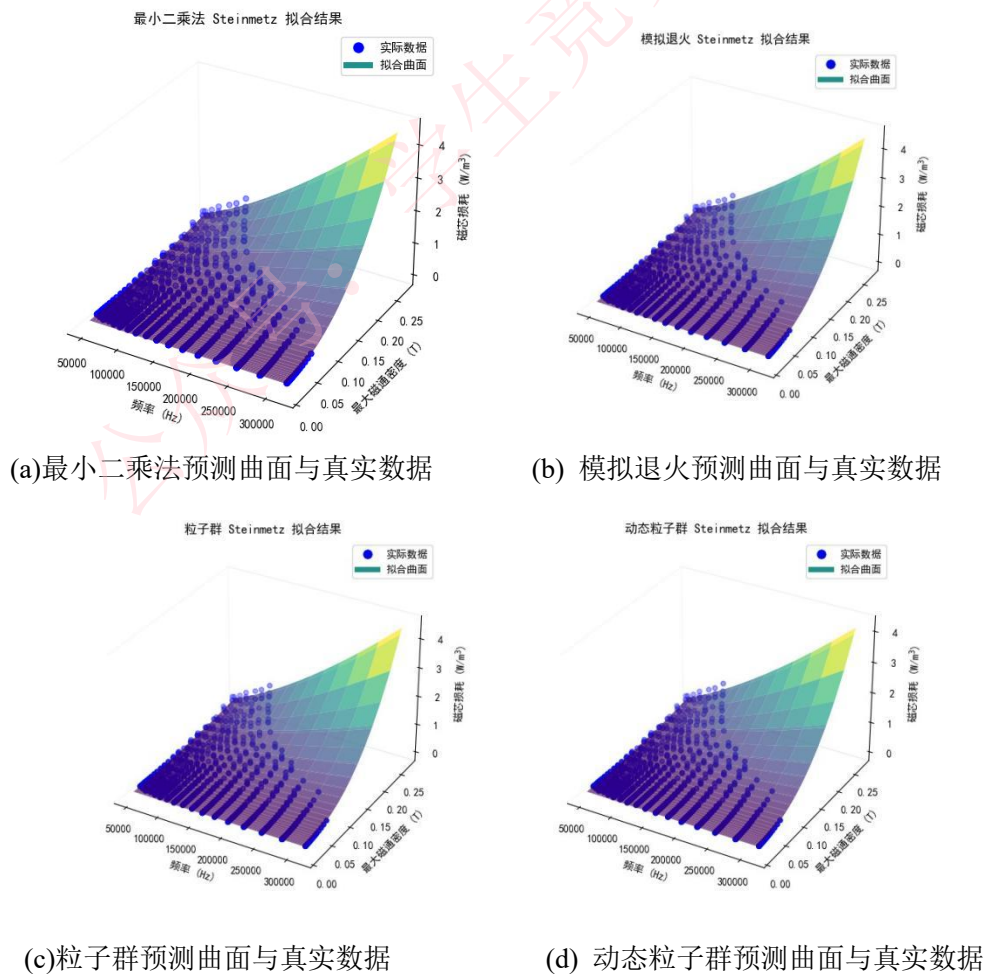
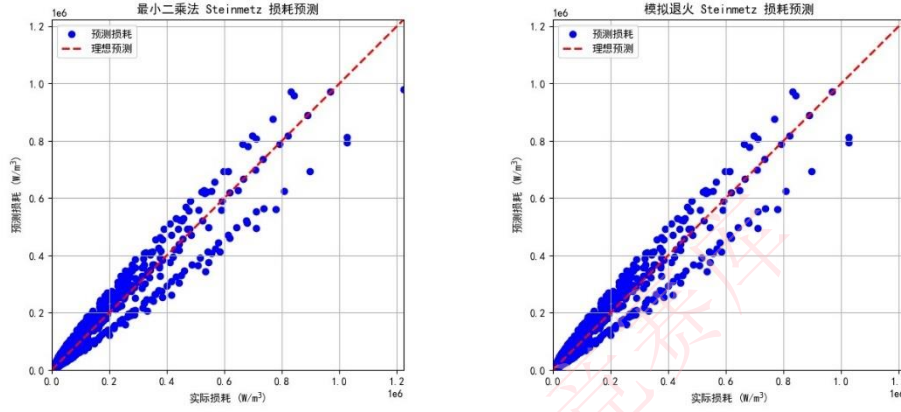


图 5.4 四种算法拟合曲面与真实数据三维图

最后，本文对最小二乘法和模拟退火算法得到的参数进行预测。由于粒子群算法及其动态版本的优化结果与前述方法相差不大，因此不再展示这些结果。我们展示了预测结果与真实值之间的差距，如图 5.5 所示。从图中可以看出，这两种方法能够很好地拟合预测损耗，但对于一些损耗较大的数据点，预测仍存在一定的误差，这可能是由于数据主要集中在较低损耗范围，导致模型学习到的参数对高损耗值的拟合能力不足。因此，模型在处理损耗较大的数据时，预测效果受到限制。



(a)最小二乘法损耗预测与真实损耗

(b) 模拟退火损耗预测与真实损耗

图 5.5 最小二乘与模拟退火损耗预测与真实损耗

5.2.5 加入温度修正的斯坦麦茨方程及其参数寻优

上一节中，本文利用四种方法拟合斯坦麦茨方程三个参数，结果表明四种方法能够较好的拟合斯坦麦茨方程的三个参数。本节中，也使用这四种方法对加入温度修正的斯坦麦茨方程进行参数寻优。

前一小节中，本文证明了温度对损耗预测有影响，但不能确定温度与损耗之间的关系，因此在本节中，测试了五种温度修正^[1,2]方法，包括线性修正、二次函数修正、三次函数修正、指数修正、幂函数修正，五种修正方法的公式如式(5.12)到式(5.16)所示。

$$P(f, B_m, T) = k \cdot f^\alpha \cdot B_m^\beta \cdot (c_0 + c_1 \cdot T) \quad (5.12)$$

$$P(f, B_m, T) = k \cdot f^\alpha \cdot B_m^\beta \cdot (c_0 + c_1 \cdot T + c_2 \cdot T^2) \quad (5.13)$$

$$P(f, B_m, T) = k \cdot f^\alpha \cdot B_m^\beta \cdot (c_0 + c_1 \cdot T + c_2 \cdot T^2 + c_3 \cdot T^3) \quad (5.14)$$

$$P(f, B_m, T) = k \cdot f^\alpha \cdot B_m^\beta \cdot e^{c_0 \cdot T} \quad (5.15)$$

$$P(f, B_m, T) = k \cdot f^\alpha \cdot B_m^\beta \cdot T^{c_0} \quad (5.16)$$

式中， T 表示温度， P 表示磁芯损耗， f 表示频率， B_m 表示磁通密度的峰值， k 、 α 、 β 、 c_0 、 c_1 、 c_2 、 c_3 是根据实验数据拟合的系数，其中 $1 < \alpha_1 < 3$ ， $2 < \beta_1 < 3$ 。

接着，本文对五种温度修正方法在斯坦麦茨方程在四种方法的基础上进行参数寻优，表 5.4 展示了参数寻优和预测结果，并把三种指标预测最好的结果进行了加粗展开。从结果中可以看出，动态粒子群算法相较于粒子群算法有所提高，在所有修正方法参数寻优中，都表现出寻优效果最好。

表 5.4 线性温度修正参数寻优预测结果

	最小二乘法	模拟退火	粒子群	动态粒子群
k	0.8103	0.3289	0.7158	0.1016
α	1.4385	1.4385	1.4383	1.4385
β	2.4324	2.4324	2.4321	2.4324
c_0	2.3093	5.6891	2.6191	18.4121
c_1	-0.012	-0.0308	-0.0141	-0.0997
R^2	0.9908	0.9908	0.9908	0.9908
RMSE	16539.8649	16539.8649	16539.8714	16539.8649
MAPE	25.6267	25.6278	25.6540	25.6267

表 5.5 二次温度修正参数寻优预测结果

	最小二乘法	模拟退火	粒子群	动态粒子群
k	0.2956	0.9733	1.8516	0.2126
α	1.4667	1.4410	1.3140	1.4671
β	2.4509	2.4340	2.3427	2.4512
c_0	5.8076	2.3134	4.6067	8.0489
c_1	-0.0721	-0.0286	-0.0564	-0.0100
c_2	0.0004	0.0002	0.0003	0.0006
R^2	0.9955	0.9954	0.9930	0.9955
RMSE	11610.6255	11695.8055	14395.6452	11610.6722
MAPE	22.1853	23.8763	35.6891	22.1620

表 5.6 三次温度修正参数寻优预测结果

	最小二乘法	模拟退火	粒子群	动态粒子群
k	0.2824	0.2025	0.0305	0.1390
α	1.4673	1.4671	1.6143	1.5331
β	2.4516	2.5255	2.4739	2.5476
c_0	6.4094	6.2701	6.9996	6.8977
c_1	-0.0993	0.0924	0.0908	-0.0976
c_2	0.0001	-0.0033	-0.0034	0.0001
c_3	-2.6814e-06	2.2240e-05	2.3715e-05	-1.4833e-06
R^2	0.9955	0.9933	0.9907	0.9948
RMSE	11564.9548	14139.1802	16625.3871	12440.3612
MAPE	22.1023	19.2200	20.3620	17.4967

表 5.7 指数温度修正参数寻优预测结果

	最小二乘法	模拟退火	粒子群	动态粒子群
k	1.8501	3.2368	3.7146	1.8496
α	1.4470	1.3944	1.3806	1.4471
β	2.4364	2.4021	2.3880	2.4364
c_0	-0.0082	-0.0082	-0.0082	-0.0082
R^2	0.9929	0.9926	0.9924	0.9929
RMSE	14530.8364	14825.0974	14991.0669	14530.8388
MAPE	24.2798	28.0222	29.6398	24.2766

表 5.8 幂级数温度修正参数寻优预测结果

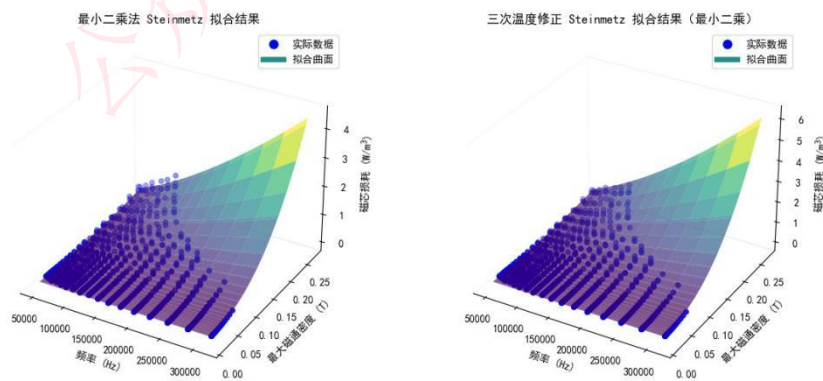
	最小二乘法	模拟退火	粒子群	动态粒子群
k	4.7384	4.8198	3.0940	4.7384
α	1.4635	1.4619	1.5066	1.4635
β	2.4485	2.4475	2.4893	2.4485
c_0	-0.4018	-0.4019	-0.4055	-0.4018
R^2	0.9953	0.9953	0.9951	0.9953
RMSE	11772.7018	11773.0303	12030.6151	11772.7018
MAPE	22.3830	22.4793	19.1747	22.3830

表 5.9 展示了所有修正方法最好的预测结果以及不加修正的预测结果对比。从表中可以看出，三次函数修正无论是 R^2 ，还是 RMSE，或 MAPE 都表现出较好的性能。这是由于三次函数具有更高的灵活性，能够更好地拟合非线性的温度-响应关系。与线性 and 二次函数相比，三次函数可以捕捉到复杂的变化趋势。指数函数虽然可以处理非线性，但是其温度寻优参数只多了一个，这可能会导致效果没有二次函数修正或三次函数修正效果好。

表 5.9 有无温度修正寻优预测结果

	无修正	线性	二次	三次	指数	幂函数
R^2	0.9448	0.9908	0.9955	0.9955	0.9929	0.9953
RMSE	40474.5	16539.8	11610.6	11564.9	14530.8	11772.7
MAPE	35.6626	25.6267	22.1620	22.1023	24.2766	22.3830

图 5.6 展示了三次修正预测结果与不加修正预测结果图，从图中可以看出，在频率较小时，拟合平面不能拟合到较大的磁芯损耗，而三次函数温度修正能够拟合到较大的磁芯损耗，因此三次函数温度修正方法使得拟合曲面更加接近真实数据，证明了所提出的三次函数方法的有效性。



(a)最小二乘法损耗预测与真实损耗 (b) 三次修正损耗预测与真实损耗(最小二乘)

图 5.6 有无温度修正损耗预测与真实损耗对比

为了更好地展示温度修正对预测结果的影响，图 5.7 呈现了在有无温度条件下真实值与预测值之间的差异。图中，红色点表示经过温度补偿后的预测磁芯损耗，而蓝色点则代表未进行温度补偿的预测结果。可以明显看出，经过温

度补偿的预测值与真实值更为接近，显示出显著的提升。这进一步验证了所采用的三次线性温度修正斯坦麦茨方程的有效性。

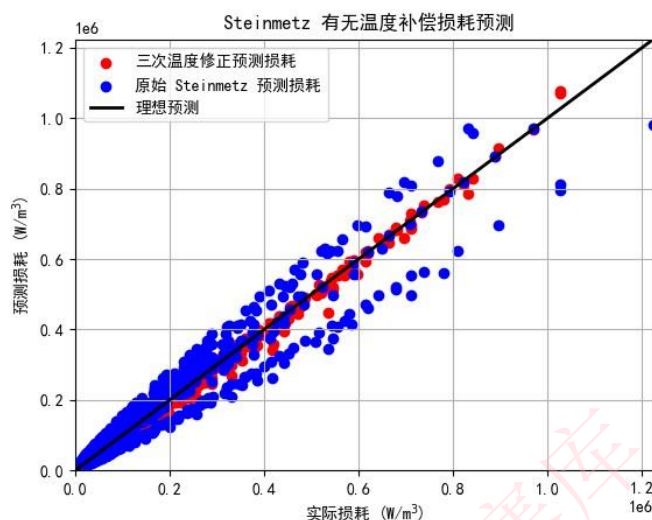


图 5.7 有无温度补偿斯坦麦茨方程损耗预测图

5.3 本章小结

本节首先使用粒子群算法对原始斯坦麦茨方程在不同温度下的三个常数参数 k_1 、 α_1 、 β_1 进行优化，结果显示温度对斯坦麦茨方程的常数影响显著，进而影响预测结果。接着，本文采用粒子群算法、模拟退火算法和最小二乘法对原始斯坦麦茨方程的常数参数进行优化，并对粒子群算法进行了改进，以提高其在原始斯坦麦茨方程中的准确性。随后，构建了几种适用于不同温度变化的磁芯损耗修正方程，包括线性、二次、三次、指数和幂级数温度修正方法。最后，对这些温度修正方程进行了四种方法的参数优化与分析，结果表明，考虑温度因素的修正方程在预测磁芯损耗方面优于原始方程。

六、问题三的模型的建立与求解

6.1 问题分析

磁芯损耗是一个核心指标，其大小直接关系到设备的效率与稳定性。为了精准提升磁性元件的性能，需要三者如何独立或协同作用于磁芯损耗，并探索实现最低损耗的最优条件。

本文分析温度、励磁波形以及磁芯材料对磁芯损耗的独立及协同作用。由于这三个因素均为离散变量，无法直接进行磁芯损耗与这些工况条件的相关性分析。首先，在单一因素下对磁芯损耗的差异性进行分析。接着，通过计算两两因素交叉分类下的磁芯损耗均值和方差，分析两因素之间的协同效应。基于以上分析结果，确定出能够使磁芯损耗最小的最佳因素组合。最后，通过三因素分类下的磁芯损耗均值和方差的热点图验证所预测的最佳因素的准确性和有效性。

6.2 模型建立与求解

6.2.1 数据提取与分布分析

首先，从数据集中提取磁芯损耗、温度、励磁波形及磁芯材料的特征数据，然后根据温度、励磁波形和磁芯材料分别对磁芯损耗进行单因素分组，并通过多因素交叉分组分析其协同作用。数据提取流程如图 6.1 所示。

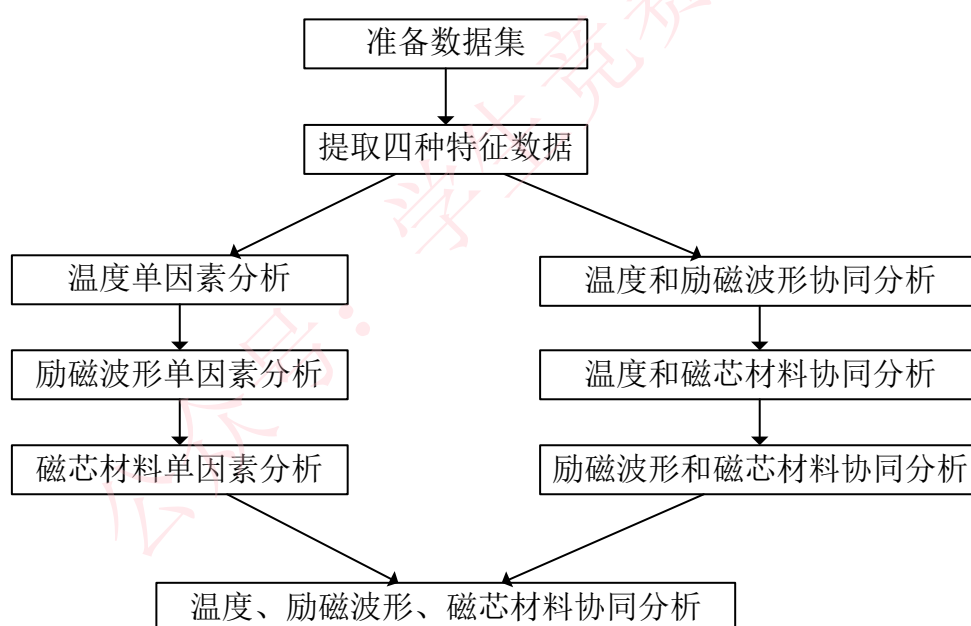


图 6.1 数据集整合流程图

首先绘制磁芯损耗的分布直方图，如图所示。观察发现磁芯损耗数据明显不符合正态分布，并使用 Kolmogorov-Smirnov 检验得到检验统计量为 0.299，p 值为 0.0 进一步验证其不符合正态分布。进一步进行了长尾分布检验，发现其偏度为 3.533，峰度为 15.899，是明显的重度右长尾分布。所以本节使用 Kruskal-Wallis H 检验进行了不同条件下的磁芯损耗差异性分析，来确定磁芯损耗和不同的工况条件间是否具有明显的相关性。

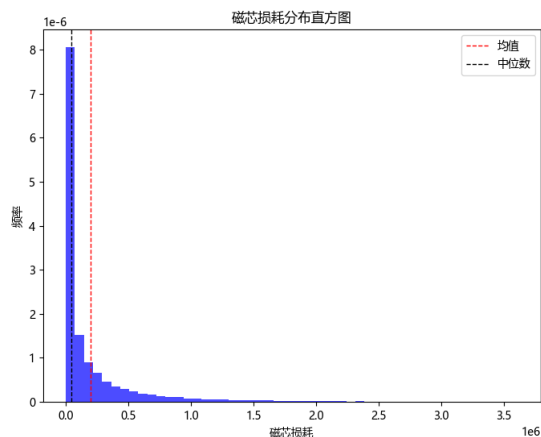


图 6.2 磁芯损耗分布直方图

6.2.2 单因素对磁芯损耗的影响分析

分别计算了 25、50、70、90 四种不同温度条件下的磁芯损耗均值和方差，如表 6.1 所示。并绘制了不同温度下的磁芯损耗的箱线图，如图 6.3 所示。随着温度的升高，磁芯损耗均值和方差逐渐降低，达到 70 度和 90 度时磁芯损耗的均值和方差趋于稳定。在不同温度下磁芯损耗具有显著的差异性， p 值 <0.05 。总体而言，不同温度下的磁芯损耗大小关系为：25 度 $>$ 50 度 $>$ 70 度 $>$ 90 度。

表 6.1 不同温度条件下的磁芯损耗均值和方差

温度	25	50	70	90	统计量	P 值
均值	2.36e+5	2.02e+5	1.79e+5	1.76e+5	67.71	<0.05
方差	1.92e+11	1.42e+11	1.13e+11	1.15e+11		

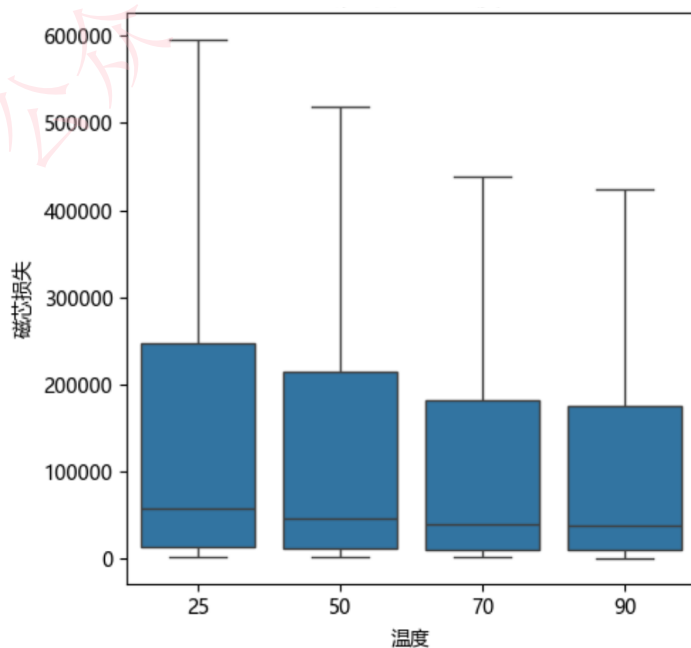


图 6.3 不同温度条件下磁芯损耗的箱线图

分别计算了正弦波、三角波、梯形波三种不同波形条件下的磁芯损耗均值和

方差，如表 6.2 所示。并绘制了不同励磁波形下的磁芯损耗的箱线图，如图 6.4 所示。正弦波下的磁芯损耗的均值和方差最小，梯形波下的磁芯损耗均值和方差最大。在不同励磁波形下的磁芯损耗具有显著的差异性， p 值 <0.05 。总体而言，不同励磁波形下的磁芯损耗大小关系为：梯形波 $>$ 三角波 $>$ 正弦波。

表 6.2 不同励磁波形下的磁芯损耗均值和方差

励磁波形	正弦波	三角波	梯形波	统计量	P 值
均值	8.76e+4	2.47e+5	2.62e+5	819.88	<0.05
方差	2.23e+10	1.89e+11	1.94e+11		

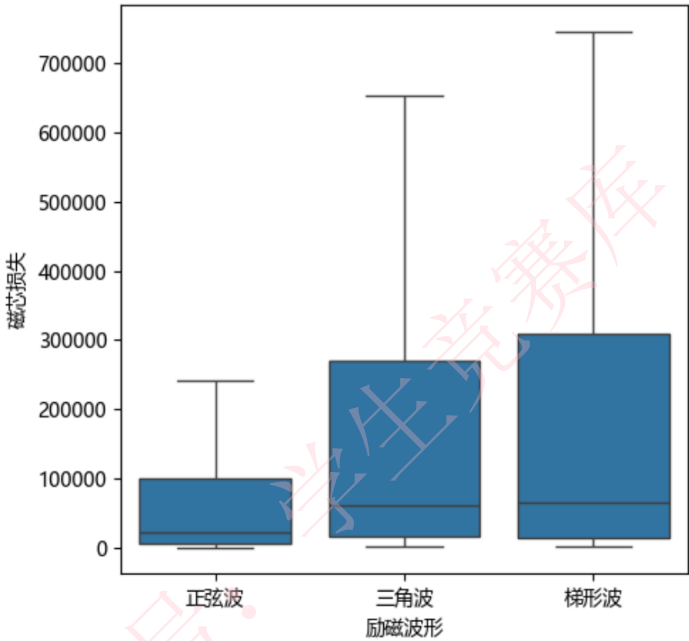


图 6.4 不同励磁波形条件下磁芯损耗的箱线图

分别计算了材料一、材料二、材料三、材料四四种不同材料条件下的磁芯损耗均值和方差，如表 6.3 所示。并绘制了不同磁芯材料的磁芯损耗箱线图，如图 6.5 所示。材料四下的磁芯损耗的均值最小和方差最小，材料三下的磁芯损耗均值和方差最大。在不同磁芯材料下磁芯损耗具有显著的差异性， p 值 <0.05 。总体而言，不同磁芯材料的磁芯损耗大小关系为：材料三 $>$ 材料二 $>$ 材料一 $>$ 材料四。

表 6.3 不同磁芯材料的磁芯损耗均值和方差

磁芯材料	材料一	材料二	材料三	材料四	统计量	P 值
均值	1.80e+5	2.34e+5	2.64e+5	1.09e+5	295.23	<0.05
方差	1.15e+11	1.67e+11	2.17e+11	4.57e+10		

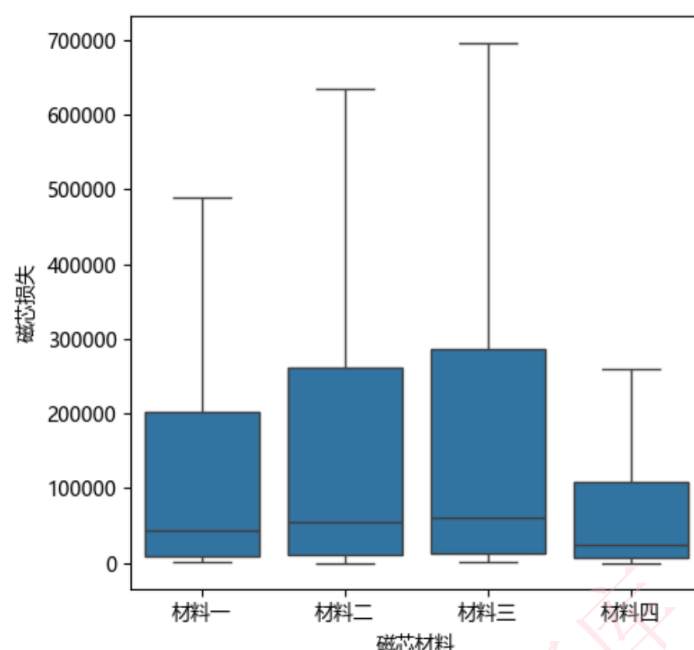


图 6.5 不同磁芯材料的磁芯损耗箱线图

6.2.3 两两因素对磁芯损耗的协同影响分析

表 6.4 计算了温度和励磁波形协同条件下的磁芯损耗均值和方差，并绘制温度和励磁波形协同条件下的磁芯损耗箱线图，如图 6.6 所示。可以看出在不同的温度条件下，正弦波均具有最小的磁芯损耗。而在不同的励磁波形下，70 度和 90 度都取得了较低的磁芯损耗均值和方差，这与 6.3.1 中的结论相符。所以，温度和励磁波形对磁芯损耗的协同影响是：在正弦波下，较高的温度可以获得更低的磁芯损耗。

表 6.4 温度和励磁波形协同条件下磁芯损耗均值及方差

励磁波形 \ 温度	指标	25	50	70	90
正弦波	均值	1.09e+5	9.03e+4	7.21e+4	7.46e+4
	方差	3.16e+10	2.33e+10	1.60e+10	1.57e+10
三角波	均值	2.85e+5	2.47e+5	2.25e+5	2.26e+5
	方差	2.55e+11	1.82e+11	1.46e+11	1.61e+11
梯形波	均值	3.39e+5	2.70e+5	2.32e+5	2.17e+5
	方差	2.78e+11	2.02e+11	1.55e+11	1.43e+11

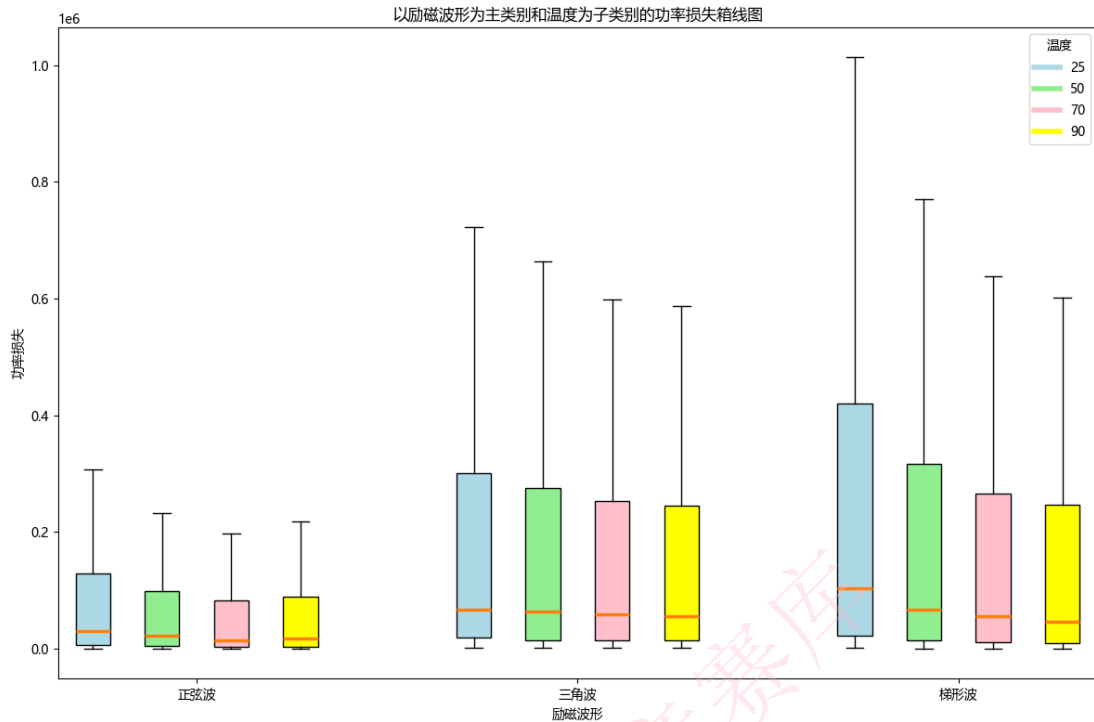


图 6.6 温度和励磁波形协同条件下的磁芯损耗箱线图

表 6.5 计算了温度和磁芯材料协同条件下的磁芯损耗均值和方差，并绘制温度和磁芯材料协同条件下的磁芯损耗箱线图，如图 6.7 所示。可以看出在材料一、材料三、材料四条件下，90 度温度具有最小的磁芯损耗均值，在材料二条件下，70 度温度具有最小的磁芯损耗均值。而在不同的温度下，材料四都取得了较低的磁芯损耗均值和方差，这与 6.3.1 中的结论相符。所以，温度和磁芯材料对磁芯损耗的协同影响是：在 90 度下，材料四可以获得更低的磁芯损耗。

表 6.5 温度和磁芯材料协同条件下磁芯损耗均值及方差

磁芯材料 \ 温度	指标	25	50	70	90
材料一	均值	2.27e+5	1.84e+5	1.55e+5	1.53e+5
	方差	1.79e+11	1.12e+11	7.72e+10	8.81e+10
材料二	均值	2.67e+5	246e+5	2.05e+5	2.15e+5
	方差	2.09e+11	1.82e+11	1.30e+11	1.43e+11
材料三	均值	3.08e+5	2.66e+5	2.46e+5	2.34e+5
	方差	2.78e+11	2.13e+11	1.88e+11	1.79e+11
材料四	均值	1..29e+5	1.05e+5	1.05e+5	9.75e+4
	方差	6.76e+10	3.83e+10	4.28e+10	3.30e+10

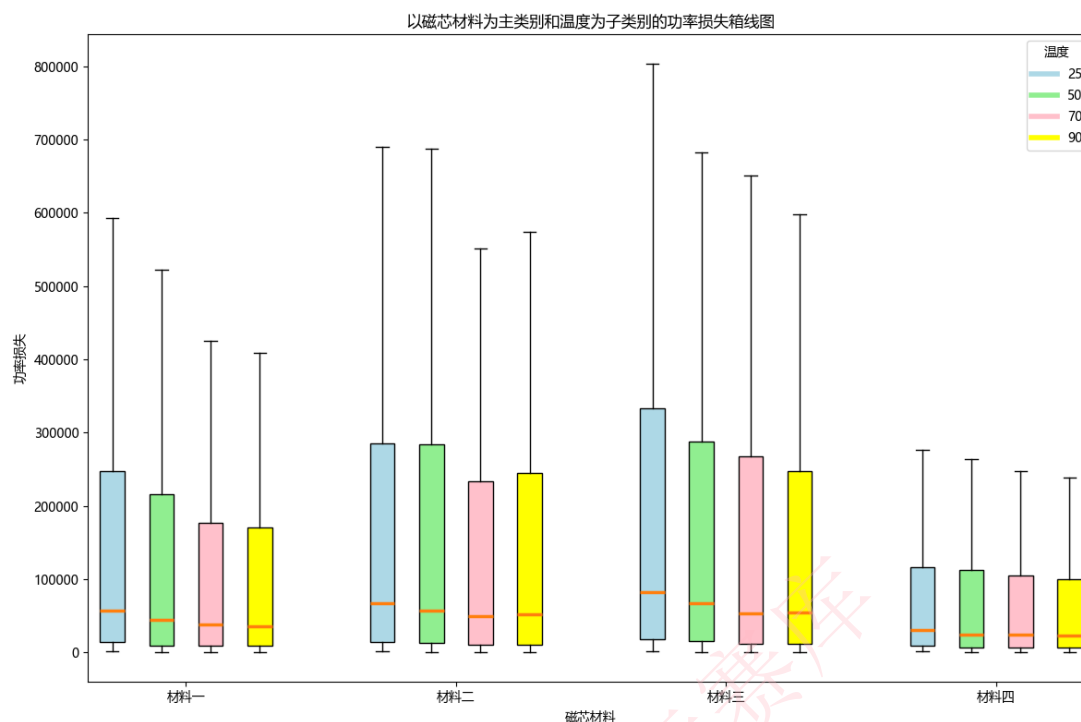


图 6.7 温度和磁芯材料协同条件下的磁芯损耗箱线图

表 6.6 计算了励磁波形和磁芯材料协同条件下的磁芯损耗均值和方差，并绘制了励磁波形和磁芯材料协同条件下的磁芯损耗均箱线图，如图 6.8 所示。可以看出在不同的励磁波形条件下，材料四均具有最小的磁芯损耗。而在不同的磁芯材料下，正弦波都取得了较低的磁芯损耗均值和方差，这与 6.3.1 中的结论相符。所以，励磁波形和磁芯材料对磁芯损耗的协同影响是：在正弦波下，材料四可以获得更低的磁芯损耗。

表 6.6 励磁波形和磁芯材料协同条件下磁芯损耗均值及方差

励磁波形 \ 磁芯材料	指标	材料一	材料二	材料三	材料四
正弦波	均值	1.02e+5	9.48e+4	1.03e+5	4.34e+4
	方差	2.97e+10	2.24e+10	2.80e+10	4.18e+09
三角波	均值	2.29e+5	3.36e+5	3.30e+5	1.41e+5
	方差	1.81e+11	2.50e+11	2.77e+11	6.57e+10
梯形图	均值	1.95e+5	2.91e+5	3.48e+5	1.35e+5
	方差	1.02e+11	2.14e+11	2.95e+11	5.01e+10

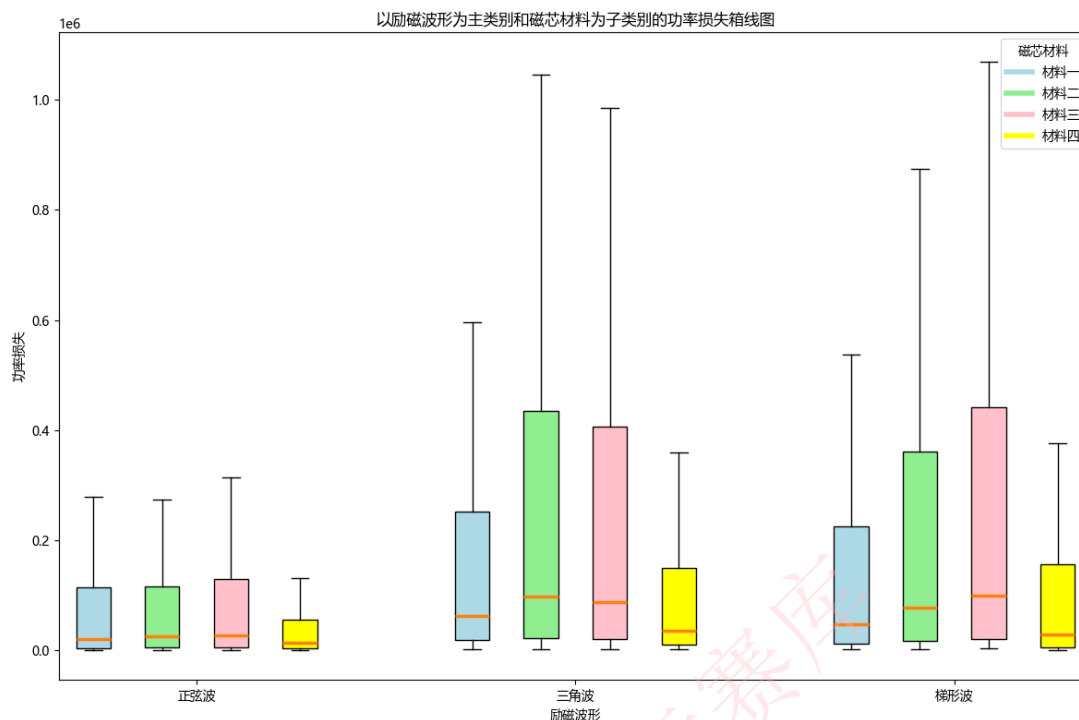


图 6.8 磁芯材料和励磁波形协同条件下的磁芯损耗箱线图

6.2.4 三因素协同影响下的最优条件分析及热点图

在上述单因素和两两因素对磁芯损耗协同影响分析中，可以发现随着温度升高磁芯损耗逐渐降低，并在 70 度和 90 度时趋于稳定。正弦波的磁芯损耗明显低于三角波和梯形波。材料四的磁芯损耗最低，且磁芯损耗方差最小。基于上述结论可以推测，在 90 度、正弦波、材料四条件下可以获得最低的磁芯损耗，损耗结果在表 6.7 中展示。

表 6.7 选择的最优条件

	温度	波形	材料类别
选择	90	正弦波	材料四

如图 6.9 和 6.10 所示，通过三因素协同影响下的磁芯损耗散点图可以清楚的发现，在 90 度、正弦波、材料四条件下磁芯损失均值和方差最小，证明了本节的推论。

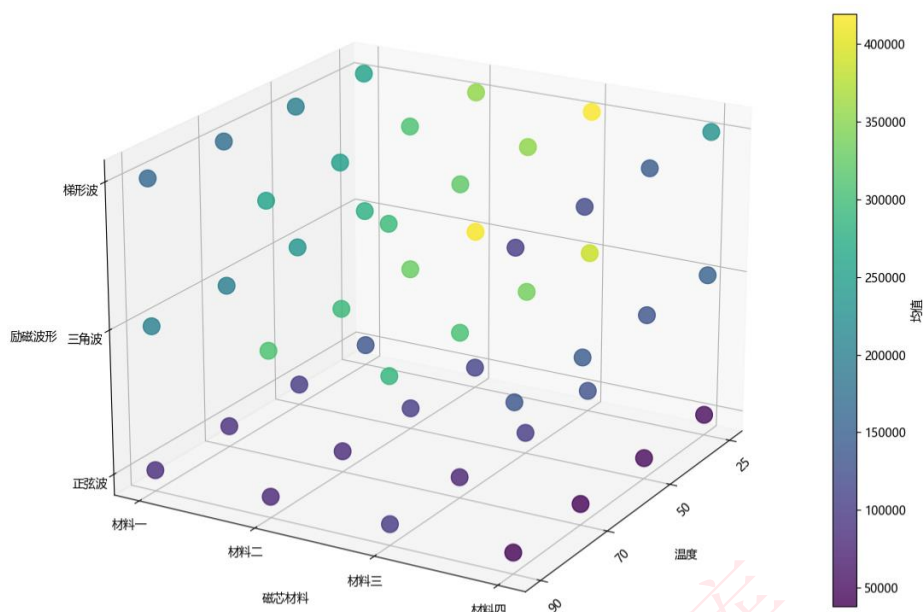


图 6.9 三因素协同条件下的磁芯损耗均值散点图

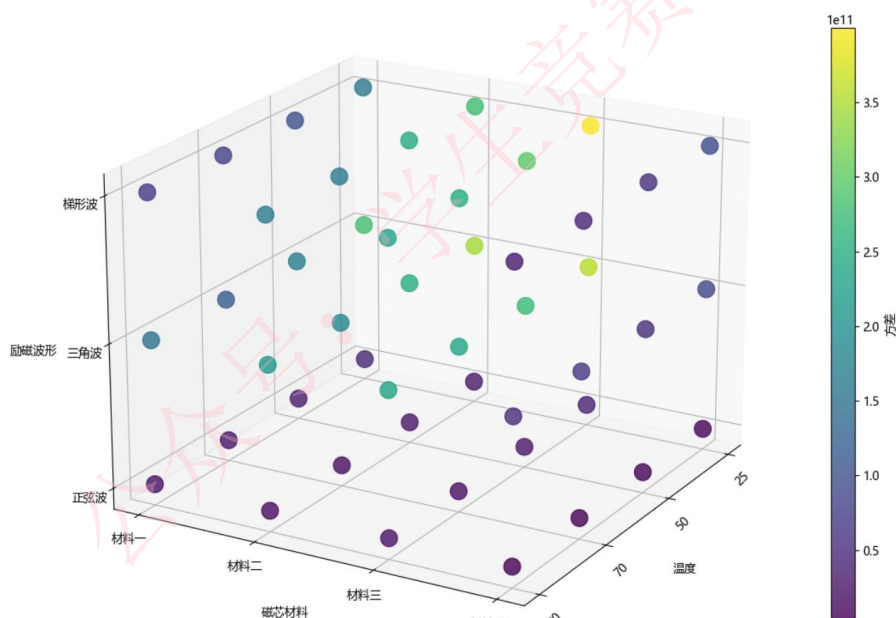


图 6.10 三因素协同条件下的磁芯损耗方差散点图

6.3 本章小结

根据问题分析，本节分别进行了温度、励磁波形和磁芯材料单因素对磁芯损耗的影响分析和两两因素对磁芯损耗的协同影响分析，发现随着温度升高磁芯损耗逐渐降低，并在 70 度和 90 度时趋于稳定，即不同温度下的磁芯损耗大小关系为：25 度>50 度>70 度>90 度。正弦波的磁芯损耗在所有因素下都明显低于三角波和梯形波，三种励磁波形下的磁芯损耗大小关系为：梯形波>三角波>正弦波。材料四的磁芯损耗最低，且磁芯损耗方差最小，四种磁芯材料的磁芯损耗大小关系为：材料三>材料二>材料一>材料四。

本节进行了两两因素对磁芯损耗的协同影响分析，发现温度和励磁波形对磁芯损耗的协同影响是：随着温度升高，各种励磁波形的磁芯损耗均明显减低，在

70 度和 90 度下，正弦波可以获得更低的磁芯损耗。温度和磁芯材料对磁芯损耗的协同影响是：随着温度升高，各种材料的磁芯损耗均明显减低，在 90 度下，材料四可以获得更低的磁芯损耗。而励磁波形和磁芯材料对磁芯损耗的协同影响是：在正弦波下，材料四可以获得更低的磁芯损耗。

所以可以推测，在 90 度、正弦波、材料四条件下可以获得最优的磁芯损耗，并在附件一数据集上通过三因素协同影响磁芯损耗三点图证明了上述推论。

公众号：学生竞赛库

七、问题四的模型的建立与求解

7.1 问题分析

在磁芯损耗研究领域，尽管存在多种传统模型，这些模型在不同条件下具备一定的应用价值，但普遍面临精度不足和适用范围受限的挑战。目前，业界缺乏一个广泛适用且能提供高精度预测的磁芯损耗模型，这降低了整体功率变换器的效率。因此，期望开发出一个可跨越不同材料类型与工况条件的磁芯损耗预测模型，提升磁性元件设计的准确性与效率。

因此，本文首先根据斯坦麦茨方程先验公式，提取了磁通密度最大值特征，并且加入了材料类型、温度、频率、波形特征，共五个特征作为磁芯损耗模型的训练集。接着，本文针对数据集提出一种特征重要性提取的双向长短时记忆回归网络对磁芯损耗进行预测，并与 CNN 网络，梯度提升决策树进行对比。最后，本文对网络预测结果进行预测精度和泛化能力分析，并给出了附件四，十个数据的预测结果。问题四求解流程图如图 7.1 所示。

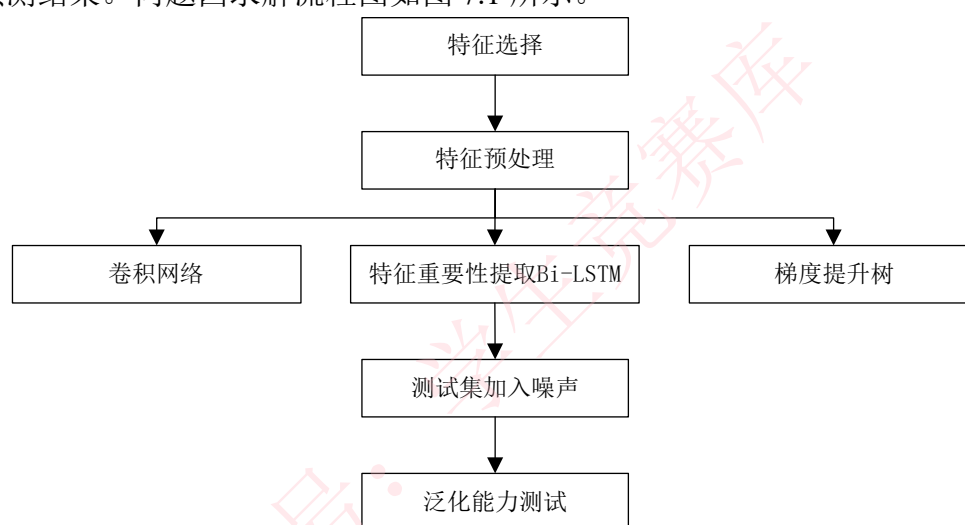


图 7.1 问题四求解流程图

7.2 模型建立与求解

7.2.1 双向 LSTM 模型

LSTM 网络是一种特殊的循环神经网络（RNN），以输入门、遗忘门和输出门等门控机制为特点，旨在克服传统 RNN 在长序列数据中的梯度消失和梯度爆炸问题，并能更有效地捕捉数据长期依赖关系。LSTM 只考虑前面时刻的信息，对信息理解不够全面，双向 LSTM 是在 LSTM 基础上发展而来，结合了前面时刻和后面时刻的信息流，能更全面的理解和分析输入序列的语义^[8]。双向 LSTM 结构图如图 7.2 所示。

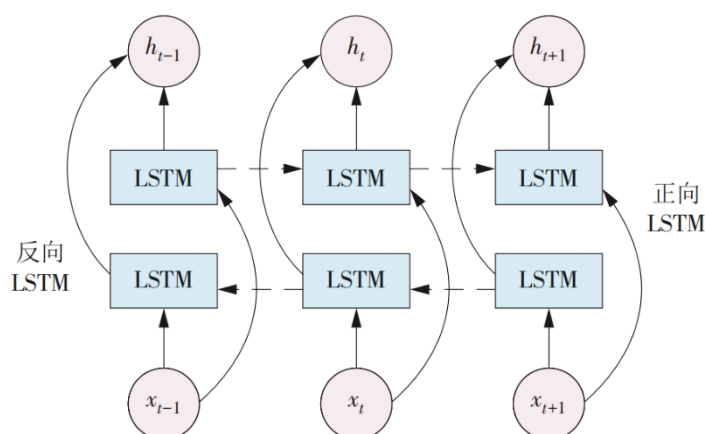


图 7.2 双向 LSTM 结构图

7.2.2 梯度提升树模型

梯度提升树^[7] (Extreme Gradient Boosting, XGBoost) 是一种高效的梯度提升决策树算法。核心是采用集成思想, 通过对原有的 GBDT 进行改进, 将多个弱学习器通过优化方法整合为一个强学习器, 即通过多棵决策树共同决策, 每棵决策树都尝试纠正前一个决策树的错误来逐步改进模型。XGBoost 支持并行和分布式计算, 大大提高了训练速度; 提供许多超参数, 如学习率、最大样本等, 进一步优化了模型性能; 支持 L1 和 L2 正则化, 可以防止过拟合和减少模型复杂度。XGBoost 模型原理图如图 7.3 所示。

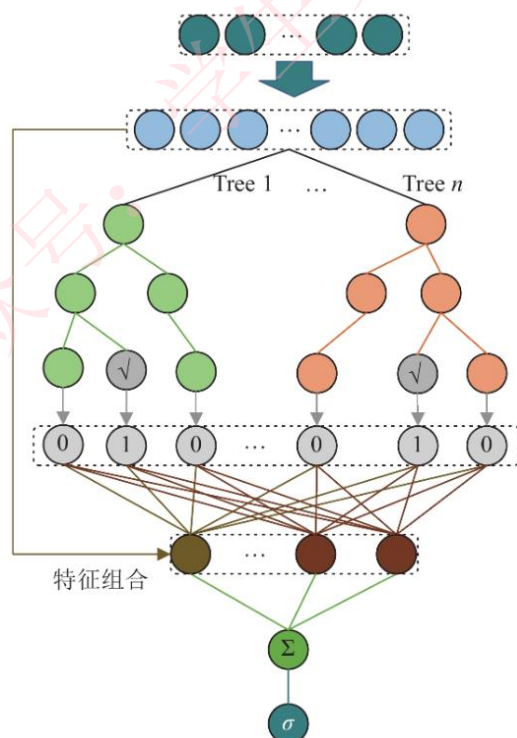


图 7.3 XGBoost 模型原理图

7.2.3 数据特征提取

首先, 对温度数据进行特征提取。温度数据在附件 1 中包含 25°C、50°C、70°C 和 90°C 四个数据点。为了进行特征提取, 我们对温度数据进行最大最小值归一化。归一化的目的是将温度与其他数据不同量纲的数据缩放到相同的范围内, 消除量纲的影响。

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}} \quad (7.1)$$

其中 x 是原始数据, $x_{\min}$ 是最小值, 这里取 25, $x_{\max}$ 是最大值, 取 90。

同理, 对频率数据也进行最大最小值归一化, 使用提供的频率最大值 500000 和最小值 50000 进行归一化。这一过程的目的是将频率数据缩放到 0 和 1 之间, 以便消除量纲的影响, 使得不同特征在模型训练中具有相同的重要性。

由于材料和波形属于类别数据, 且它们的量纲不同, 因此需要对这两个特征进行独热编码 (One-Hot Encoding)。独热编码将每个类别转换为一个二进制向量, 以便模型能够更好地理解这些非数值特征。具体的编码关系如式 (7.2) 和式 (7.3) 所示。通过这种方式, 模型能够有效地处理材料和波形的不同类别, 避免了数字编码可能引入的顺序关系。

$$\begin{cases} \text{正弦波}:[1,0,0] \\ \text{三角波}:[0,1,0] \\ \text{梯形波}:[0,0,1] \end{cases} \quad (7.2)$$

$$\begin{cases} \text{材料1}:[1,0,0,0] \\ \text{材料2}:[0,1,0,0] \\ \text{材料3}:[0,0,1,0] \\ \text{材料4}:[0,0,0,1] \end{cases} \quad (7.3)$$

对于磁通密度特征, 斯坦麦茨方程的经验公式指出, 最大值与其他参数的相关性最强, 并且磁通密度的最大值能够有效代表磁场强度的极限特征, 直接影响材料的性能和行为。因此, 本文选择磁通密度的最大值作为网络输入特征, 通过仅使用最大值, 可以减少输入特征的维度, 简化模型的复杂性。

对于磁芯损耗值, 由于损耗值数量级相差比较大、分布不均衡, 磁芯损耗分布如图 7.4 所示, 从图中可以看出数据分布属于长尾性数据。本文采用 Kolmogorov-Smirnov (K-S) 检验对数据是否属于长尾分布进行检验, K-S 检验如式(7.4)所示。

$$D = \max_x |F_n(x) - F(x)| \quad (7.4)$$

式中, $F_n(x)$ 是样本的经验分布函数, $F(x)$ 是理论分布函数。

经过计算, K-S 检验统计量为 0.2988, 显著大于 0.05, 表明磁芯损耗数据符合长尾分布。长尾分布通常意味着数据存在较大的极端值或偏态, 这可能影响模型的训练效果和预测准确性。因此, 本文对磁芯损耗进行了自然对数

($\ln$) 变换。该变换能够有效减小数据的偏态, 使其更接近正态分布, 增强分布的均匀性, 并减少极端值对数据的影响。这些变化有助于提高后续网络的训练效率, 减少训练过程中的波动, 从而提升模型的稳定性和预测性能。如图 7.5 所示, $\ln$ 变换后的磁芯损耗分布直方图显示出更好的分布特性, 为后续

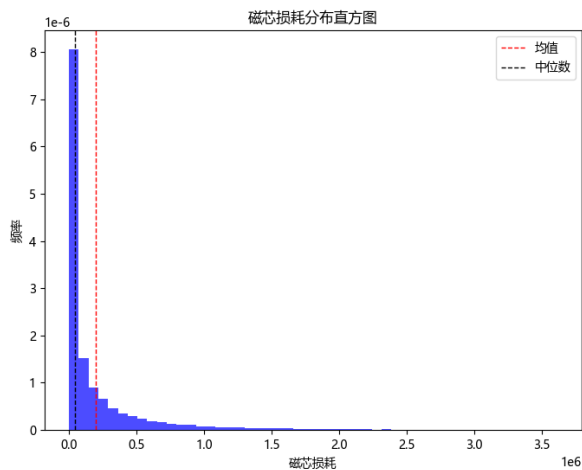


图 7.4 磁芯损耗分布直方图

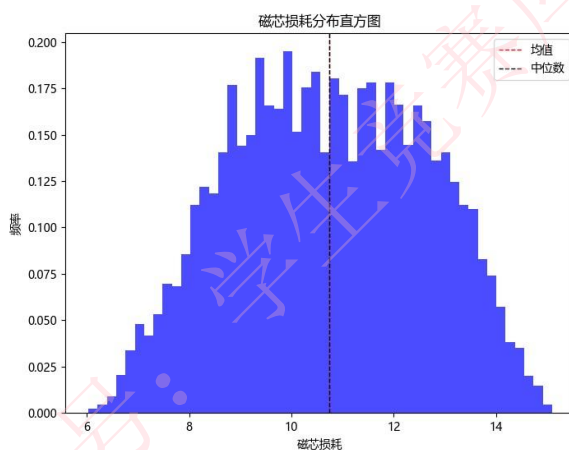


图 7.2 处理后磁芯损耗分布直方图

最后，本文将提取出的特征进行拼接，形成一个 10 维的特征向量，其格式为 [密度、温度、频率、材料、波形]。为了进行模型训练，数据集按照比例 8:1:1 划分为训练集、验证集和测试集，分别包含 9920、1240 和 1240 个样本。

7.2.4 特征重要性提取的双向长短时记忆回归网络

由于数据输入维度较少，且特征重要性提取较为复杂，针对这个问题，本文设计了一种特征重要性提取的双向长短时记忆回归网络，网络结构如图 7.6 所示。

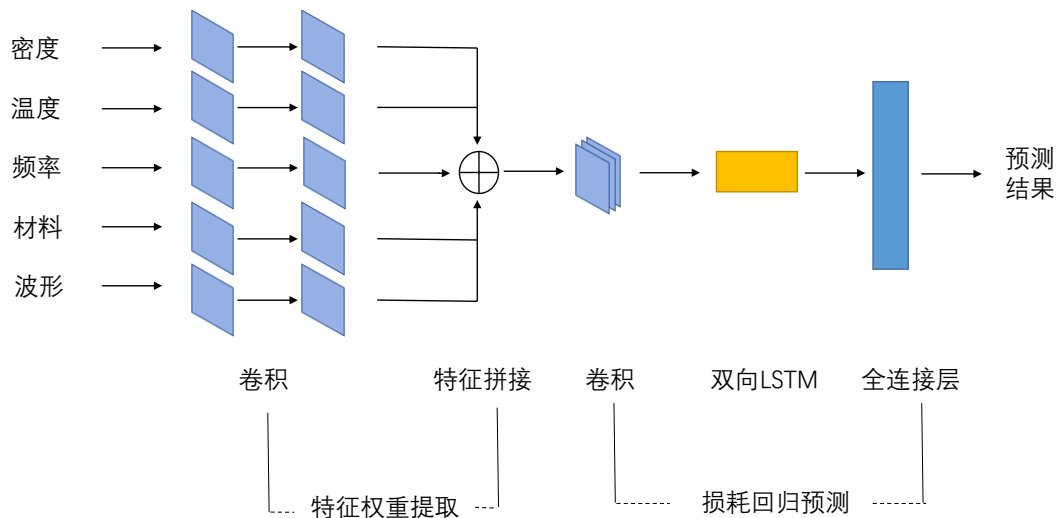


图 7.6 特征重要性提取的双向长短时记忆回归网络

网络一共包含五个部分，第一部分为对每个输入特征进行一维卷积，提取每个特征的权重；第二部分为特征拼接，将提取权重后的特征进行拼接作为后续网络输入；第三部分为卷积层，提取网络的空间特征；第四部分为双向 LSTM，双向 LSTM 允许模型从两个方向（过去和未来）理解输入特征，提高模型的泛化能力；第五部分为全连接层，输出预测的磁芯损耗值。

接着，本文对提出的模型的性能进行测试，并与传统的卷积网络模型、梯度提升树模型在验证集上进行了磁芯损耗预测对比。预测结果在决定系数、均方根误差、平均绝对百分比误差三个维度上进行了对比。结果如表 7.1 所示。

表 7.1 三种方法磁芯损耗预测

	梯度提升树	卷积网络	特征重要性提取 LSTM
R^2	0.9950	0.9837	0.9980
RMSE	27577.66	61719.31	22533.60
MAPE	8.6191	24.11	8.449

从表 7.1 中可以看出，本文所提出的网络拥有最好的测试集预测结果，使用原始的卷积网络 R^2 为 0.9837，而本文所提网络为 0.9980，远大于传统的卷积网络，证明所提出的网络拥有更强的特征提取能力，上下文信息的学习能力

接着，本文对所提网络预测结果进行可视化。可视化结果如图 7.7 所示。从图 7.7 中可以看出，特征重要性提取的双向长短时记忆回归网络能够很好的预测磁芯损耗。

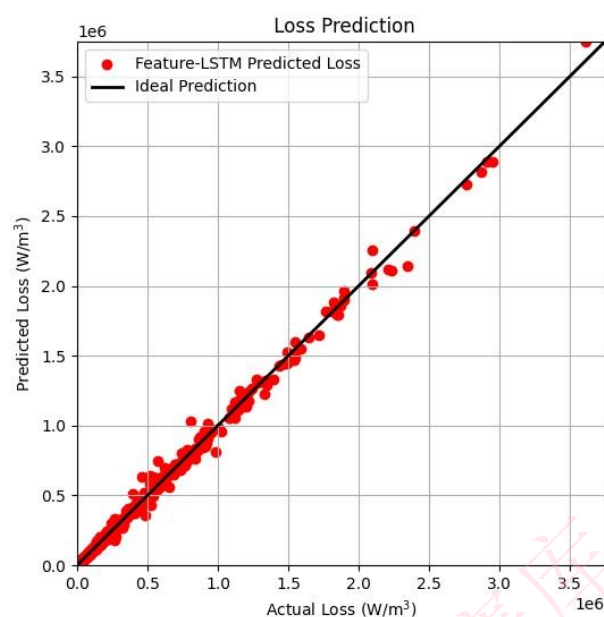


图 7.7 特征重要性提取的双向长短时记忆回归网络预测结果

然后，本文对三种方法的预测对比进行了可视化，如图 7.8 所示。图中，蓝色为 CNN 的预测结果，黄色为 XGBoost 的预测结果，红色为特征重要性提取的双向长短时记忆回归网络预测结果，可以看出，红色更加接近理想预测。而蓝色的对于一些数据点预测的结果较差，说明传统的神经网络对特征提取的能力还不足。

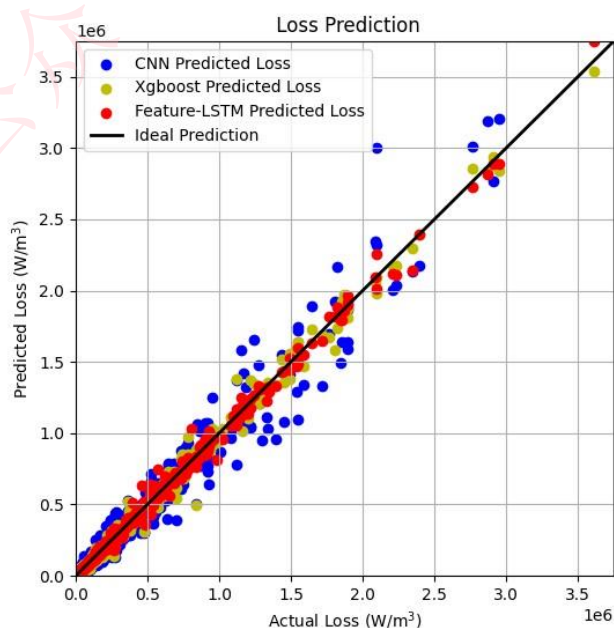


图 7.8 三种预测结果对比

最后，本文对特征重要性提取的双向长短时记忆回归网络模型的泛化能力进行测试，在模型训练时，本文使用划分好的训练集和验证集，接下来使用训练好的模型进行测试集的测试，检测模型的泛化能力。并且对训练集中的温度、频率、最大密度加入标准差为 0.1 随机高斯噪声，测试模型的泛化能力。测试结果如表 7.2 所示。

表 7.2 三种方法磁芯损耗预测

	验证集	测试集	加入噪声的测试集
R^2	0.9980	0.9956	0.9952
RMSE	22533.60	24625.48	25679.60
MAPE	8.449	8.594	8.629

从表 7.2 的三个评价指标结果，在测试集上的测试结果稍逊与验证集上测试结果，加入噪声的测试集结果稍逊与不加噪声的测试集。可能的原因有测试集和验证集可能存在一定的数据分布差异，导致模型在验证集上表现较好，而在测试集上效果下降，而加入噪声后的测试集结果略有下降表明噪声对模型的预测能力产生了负面影响，但下降的数值很小，并且下降后的三个指标仍比使用梯度提升树和传统卷积网络预测效果好。因此，仍能说明本文所提出的模型泛化能力强。

图 7.9 展示了测试集结果与加入噪声的测试集的对比结果，从图中可以看出，模型在测试集上也有比较好的磁芯损耗预测结果，并且测试集加入噪声后与不加噪声的结果差异不大，证明了方法的泛化能力。

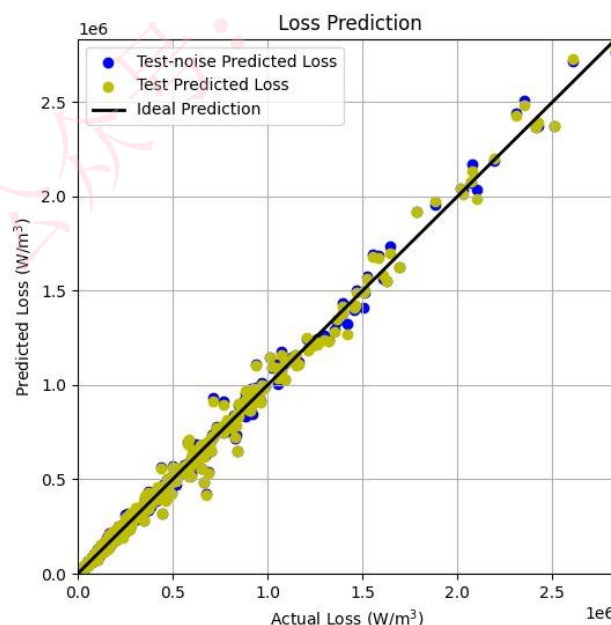


图 7.9 测试集有无噪声预测结果对比

最后，本文对附件三中样本的磁芯损耗进行了预测，表 7.3 展示了附件三中样本序号为：16、76、98、126、168、230、271、338、348、379 的磁芯损耗预测结果，保留一位小数。

表 7.3 十个样本序列的磁芯损耗测结果

样本序号	预测结果 (T)
16	1012.1
76	1483766.5
98	12225.0
126	1858.0
168	97589.4
230	76859.7
271	1876748.3
338	17495.8
348	865953.1
379	1525.8

7.3 本章小结

本节旨在开发出一个能够跨越不同材料类型与工况条件的磁芯损耗预测模型，提升磁性元件设计的精确性与效率，为电力电子技术的进一步发展奠定坚实基础。

本文首先根据斯坦麦茨方程先验公式，提取了磁通密度最大值特征，并且加入了材料类型、温度、频率、波形特征，共五个特征作为磁芯损耗模型的训练集。接着，本文针对数据集提出一种特征重要性提取的双向长短时记忆回归网络对磁芯损耗进行预测，并与 CNN 网络，梯度提升决策树进行对比。最后，本文对网络预测结果进行预测精度和泛化能力分析。

八、问题五的模型的建立与求解

8.1 问题分析

在磁性元件的设计与优化领域内，磁芯损耗固然是一个不容忽视的核心评价指标，但在工程实践中，为了实现磁性元件整体性能的卓越与最优化，需要综合考虑多个评价指标，其中，传输磁能就是重要的评价指标之一。因此，同时考虑磁芯损耗与传输磁能这两个评价指标，求解其优化问题具有重要的理论及实践意义。

因此，本文以温度、频率、波形、磁通密度峰值及磁芯材料作为决策变量，以问题四提出的磁芯损耗预测模型和传输磁能作为目标函数，通过分析决策变量的约束范围，最终构建了一个多目标优化问题，旨在确定在什么温度、频率、波形、磁通密度峰值及磁芯材料条件下能够实现最小的磁芯损耗和最大的传输磁能。进一步地，本文针对多目标优化问题采用了两种解决方法：第一种是通过加权方式将多目标优化问题转化为单目标优化问题，并使用粒子群算法和遗传算法两种方法进行求解；第二种是通过智能优化算法 NSGA-II 和 MOPSO 直接对多目标问题进行求解，以获取 Pareto 最优解，并最终绘制 Pareto 前沿图。问题五的模型流程图如图 8.1 所示。

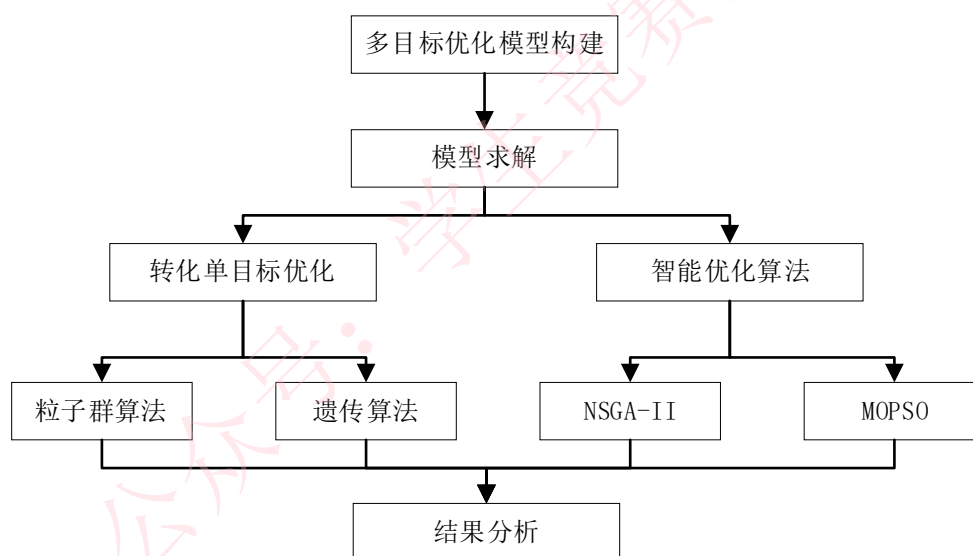


图 8.1 问题 5 模型建立流程图

8.2 模型建立与求解

8.2.1 决策变量数据整合及目标函数建立

根据分析可知，该优化问题中包含温度、频率、波形、磁通密度峰值及磁芯材料五个决策变量。结合附件中的数据，首先对决策变量的范围进行整合，其中，对于波形，存在三种波形，分别为正弦波、三角波、梯形波；对于磁芯材料，存在四种磁芯材料，分别为磁芯材料 1、磁芯材料 2、磁芯材料 3、磁芯材料 4；对于温度，存在四个温度值，分别为 25℃、50℃、70℃、90℃；对于频率，由题目可知，取值范围为 50000—500000Hz；对于磁通密度峰值，对附件 1 中的磁芯损耗数据进行处理，最终得到磁通密度峰值取值范围为 0.0096-0.3133T。最终获取所有决策变量的取值范围，即所求优化问题的决策变量约束条件。所有决策变量的取值范围如表 8.1 所示。

表 8.1 决策变量取值范围

决策变量	取值范围
温度	25、50、75、90 (°C)
频率	50000-500000 (Hz)
波形	正弦波、三角波、梯形波
磁通密度峰值	0.0096-0.3133(T)
磁芯材料	材料 1、材料 2、材料 3、材料 4

本任务旨在解决多目标函数的优化问题，需要求解温度、频率、波形、磁通密度峰值及磁芯材料在什么条件下获得最小磁芯损耗以及最大传输磁能。对于磁芯损耗，以第四问建立的磁芯损耗预测模型为第一个目标函数 F_s ；对于传输磁能，以题中给出的决策变量频率和磁通密度峰值的乘积($F_e=f \times B_m$)为第二个目标函数。

通过上述分析，最终建立了以温度、频率、波形、磁通密度峰值及磁芯材料为决策变量，最小化磁芯损耗和最大化传输磁能为目标函数的多目标优化问题，结合上述分析的约束条件，最终建立多目标优化问题的目标函数如式(8.1)所示。

$$\begin{cases} \min F_s \\ \max F_e \end{cases} \quad (8.1)$$

其中 F_s 表示磁芯损耗， F_e 表示传输磁能。

为了简化优化问题的表达，对于波形，我们定义正弦波为 1，三角波为 2，梯形波为 3；对于磁芯材料，我们定义磁芯材料 1 为 1，磁芯材料 2 为 2，磁芯材料 3 为 3，磁芯材料 4 为 4。最终建立多目标优化问题的约束：

$$\text{s.t.} \begin{cases} T \in \{25, 50, 70, 90\} \\ W \in \{1, 2, 3\} \\ L \in \{1, 2, 3, 4\} \\ f \geq 50000 \\ f \leq 500000 \\ B_m \geq 0.0096 \\ B_m \leq 0.3133 \end{cases} \quad (8.2)$$

上式中， T 表示温度， W 表示波形， L 表示磁芯材料， f 表示频率， B_m 表示磁通密度峰值。

针对上述多目标优化问题，首先对两个目标函数分析，结合经验公式分析可得，两个目标函数的量纲不一致且存在对抗关系，因此采取两种方式对多目标问题进行转化求解。

8.2.2 多目标转化单目标优化求解

首先针对目标函数量纲不一致的问题，对两个目标函数进行归一化处理，由于问题中并未给出两个目标函数的权重，设置磁通损耗权重为 a ，则传输磁能的权重为 $(1-a)$ ，将 a 作为决策变量加入最优化问题中，最终将多目标优化问题转

化为如下单目标优化问题：

$$\min(a \cdot F_s - (1-a) \cdot F_e) \quad (8.3)$$

$$\text{s.t.} \begin{cases} T \subset \{25, 50, 70, 90\} \\ W \subset \{1, 2, 3\} \\ L \subset \{1, 2, 3, 4\} \\ f \geq 50000 \\ f \leq 5000000 \\ B_m \geq 0.0096 \\ B_m \leq 0.3133 \\ a \geq 0 \\ a \leq 1 \end{cases} \quad (8.4)$$

对于上述多目标优化加权转化为单目标优化的方式，本文采用遗传算法和粒子群算法两种算法对转化后的单目标优化问题进行求解。遗传算法^[4](Genetic Algorithm, GA)是一种通过模拟自然选择和进化过程来搜索最优解的方法。该算法通过选择、交叉和变异等操作，模拟生物种群的进化机制，通过迭代优化逐步提升个体的适应度，以找到最优解。

遗传算法在解决多目标、高维度、非连续性以及非线性问题方面具有显著优势。其基本过程主要包括：参数编码、初始群体的设定、适应度函数的设计、遗传操作设计、控制参数的设定。遗传算法的基本步骤如下图 8.2 所示。

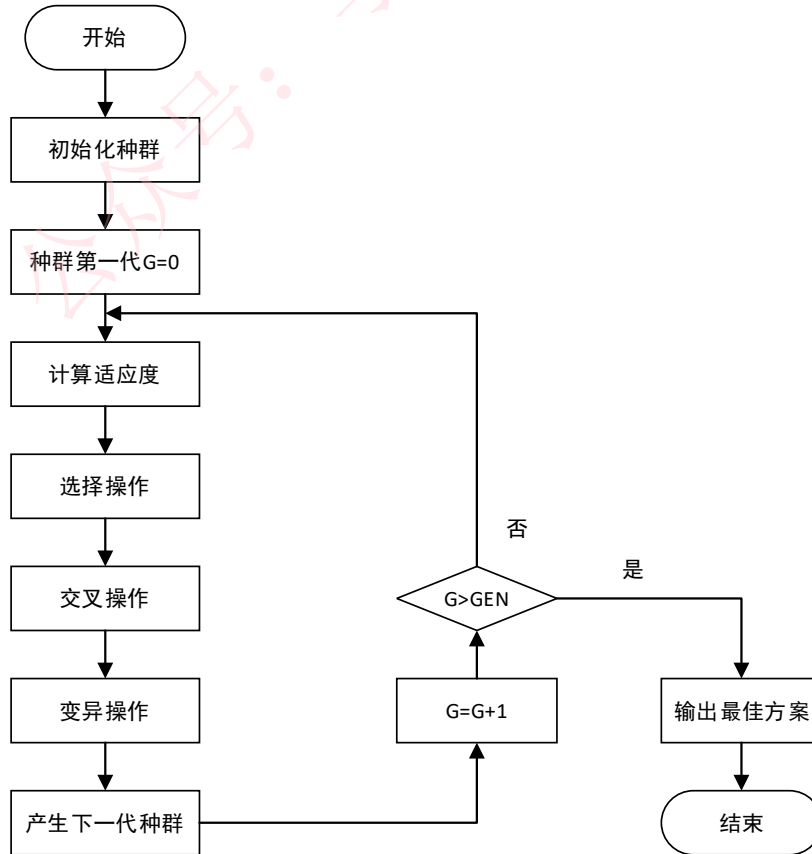


图 8.2 遗传算法流程图

粒子群算法（Particle Swarm Optimization, PSO）是一种通过模拟鸟群觅食行为而发展起来的一种基于群体协作的随机搜索算法。该算法初始化一群随机粒子，通过迭代找到最优解，每一次迭代是跟踪速度和位置极值来更新自己，在解决大数据、复杂度高、目标函数复杂的问题中具有显著优势。

接着，本文采用上述两种优化算法对多目标转化后的单目标优化问题进行求解，两种优化算法最终的求解结果如表 8.2 所示。

表 8.2 两种算法最优化求解结果

	遗传算法	粒子群算法
温度	90	90
频率	79680	79470
波形	正弦波	正弦波
磁通密度峰值	0.021	0.0190
磁芯材料	材料二	材料二
a	0.83	0.79
磁通损耗 F_s	426.3647	415.6131
最大传输磁能 F_e	1673.28	1509.93
$aF_s - (1-a)F_e$	4.6918	30.8623

通过对表格数据分析可知，遗传算法求得的目标函数最优解明显优于粒子群算法求得的最优解，比较两种算法的运行时间，遗传算法的运行时间要大于粒子群算法的运行时间，说明粒子群算法获取的解为局部最优。通过最终两个算法求得的目标函数值的对比，遗传算法求得的解优于粒子群算法求得的解，故取遗传算法的解为转化单目标后求得的最优解。对结果的实际意义分析可知，对于电力电子器件，相较于最大传输磁能，其更关注磁通损耗特性，故求得的最优解具有一定意义。

8.2.3 NSGA-II 算法

对于上述原始的多目标优化求解问题，本节采用 NSGA-II^[3]、多目标粒子群智能算法进行求解。

NSGA 算法是在简单遗传算法的基础上进行改进的，其核心思想是引入了 Pareto 最优的概念。NSGA-II 算法是对 NSGA 的改进版本。NSGA-II 算法广泛应用于多目标优化领域，旨在有效寻找多个目标函数的最优解集。采用了快速非支配排序策略和精英策略，并用拥挤度函数替代共享函数，从而显著减少了计算时间和复杂性。

NSGA-II 算法的主要步骤包括：初始化种群、评估适应度、快速非支配排序、选择、交叉与变异、合并父代和子代、更新种群。该算法通过在后代中持续筛选最佳个体，获取每个优化目标函数的最小解。其关键步骤包括快速非支配排序和拥挤度排序。

NSGA-II 算法的基本步骤如下：

步骤 1：设置算法参数，包括终止条件、种群数量、交叉率、变异率以及决策变量的约束条件。

步骤 2：根据决策变量的上下限初始化种群 P_t ，并设定 $t=0$ 。

步骤 3：计算每个个体的适应度函数值。

步骤 4：进行非支配排序后，使用交叉和变异算子生成子代种群 Q_t ，并计算子代中每个个体的适应度函数值。

- 步骤 5: 将父代和子代合并为种群 R_t , 即 $R_t = P_t \cup Q_t$ 。
- 步骤 6: 将代数更新为 $t=t+1$, 从第二代开始继续合并父子代种群。
- 步骤 7: 对 R_t 中的每个个体计算适应度函数值, 并进行快速非支配排序及拥挤距离的计算。
- 步骤 8: 采用精英选择策略, 选择前 N 个个体作为新一代个体。

快速非支配排序将种群中的个体划分为不同的等级。个体 A 支配个体 B 的条件为: A 在所有目标上优于 B, 或者在至少一个目标上优于 B, 并且在其他目标上不劣于 B。通过非支配排序, 可以形成多个 Pareto 前沿 (Pareto Front)。拥挤度距离用于评估个体在 Pareto 前沿中的稠密程度, 如式(8.4)所示。

$$G(i) = \sum_{m=1}^M (f_m^{(i+1)} - f_m^{(i-1)}) \quad (8.4)$$

其中 $G(i)$ 为个体 i 的拥挤度距离, $f_m^{(i)}$ 为第 m 个目标函数在第 i 个个体上的值。

在精英保留策略中, 首先将子代和父代合并, 然后进行快速非支配排序和拥挤度值计算, 选择等级优先级较高的个体; 在等级相同时, 选择拥挤度值较大的个体。最后, 将选择出的个体进行精英保留, 形成新的种群。精英选择的步骤如图 8.3 所示。

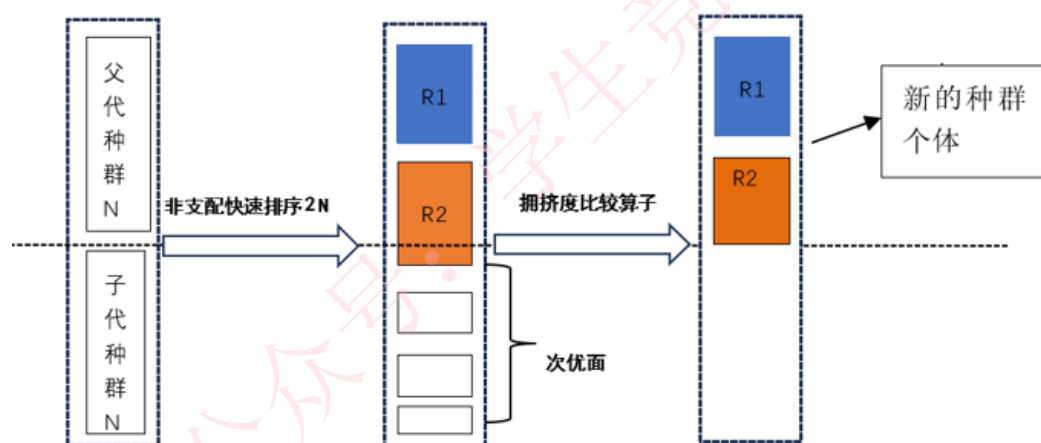


图 8.3 精英选择步骤图

多目标粒子群优化算法 (Multi-objective Particle Swarm Optimization, MOPSO) 是一种基于粒子群优化的多目标优化算法, 旨在同时优化多个相互冲突的目标函数。MOPSO 核心是通过粒子群的协作和信息共享来探索多目标解空间, 最终找到 Pareto 最优解。

主要步骤为: 初始化, 随机生成粒子的位置和速度; 评估粒子, 计算每个粒子的目标函数值, 并进行非支配排序; 更新个人最优和全局最优, 更新每个粒子的个人最优位置和全局最优位置, 考虑非支配解和拥挤度; 速度和位置更新直至最终迭代结束。MOPSO 流程图如图 8.4 所示。

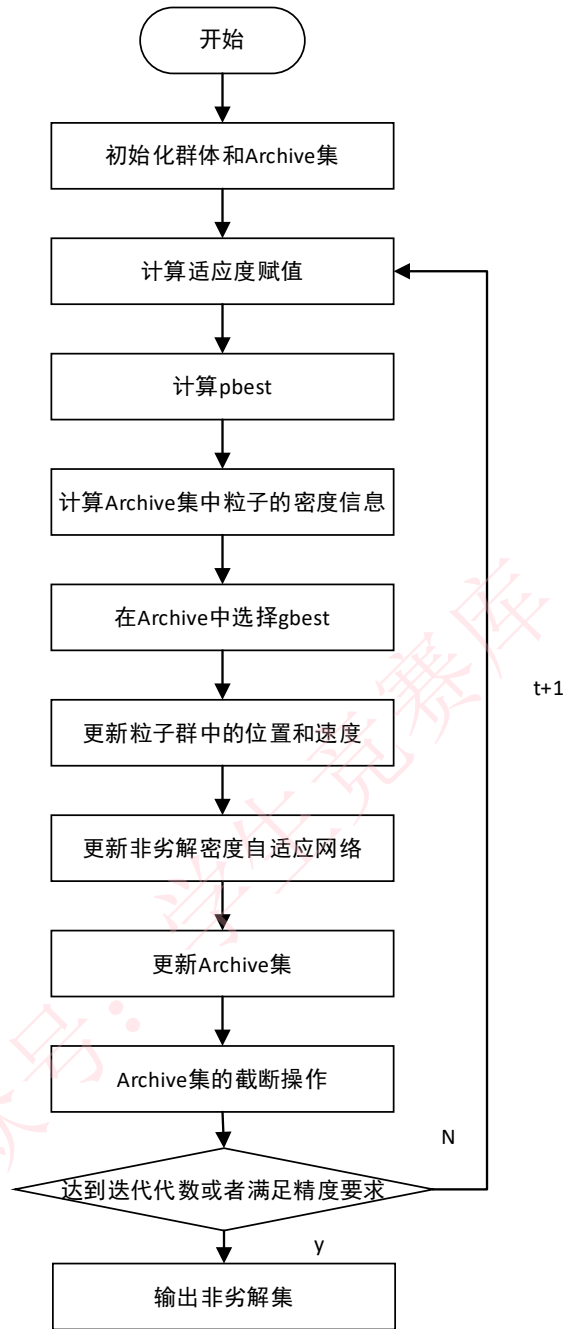


图 8.4 MOPSO 流程图

Pareto 最优是指资源分配的一种理想状态，最早使用在经济学中，具体定义为设 $x^* \in D$ ，对于任意的 $x \in D, k \in 1, \dots, K$ ，都有 $f_k(x^*) \leq f_k(x)$ 。所有的 Pareto 最优解的集合称为 Pareto 最优解集。Pareto 最优解集在空间的表现形式即为 Pareto 前沿。

接着，本文采用上述 NSGA-II 算法和 MOPSO 算法对 6.2.3 节中建立的多目标优化问题进行求解，共求得 97 个 Pareto 最优解，两种算法分别求解的最优解示例如表 8.3 所示。

表 8.3 两种算法求解最优解

	温度	频率	波形	磁芯材料	磁通密度峰值	磁芯损耗	传输磁能
NSGA-II Pareto 最优解 1	90	192418	正弦波	材料 1	0.0104	1049	2008
NSGA-II Pareto 最优解 2	90	379324	正弦波	材料 1	0.0776	143186	29439
MOPSO Pareto 最优解 1	90	470110	正弦波	材料 4	0.4261	87042	20032
MOPSO Pareto 最优解 2	90	49910	正弦波	材料 1	0.1677	741841	83703

NSGA-II 求得的 Pareto 最优解绘制的 Pareto 前沿可视图如图 8.5 所示。

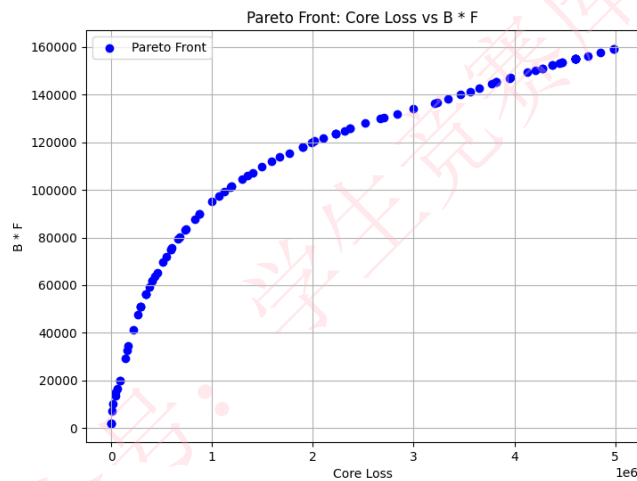


图 8.5 NSSGA-II 算法 Pareto 前沿图

MOPSO 算法求得的 Pareto 最优解绘制的 Pareto 前沿图如图 8.6 所示。

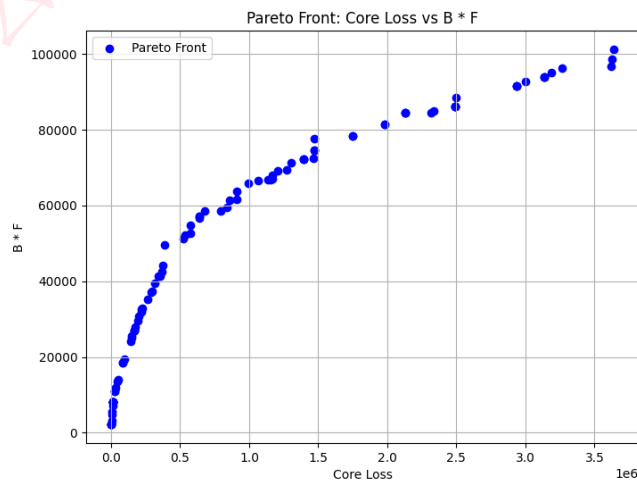


图 8.6 MOPSO 算法 Pareto 前沿图

从两个算法绘制出的 Pareto 前沿图可视化结果可以看出，NSSGA-II 算法求得的 Pareto 最优解较为集中，相邻解集之间距离近，MOPSO 算法求出的 Pareto

最优解存在较多离散点，收敛效果较差，由此可以得出 NSSGA-II 算法求得 Pareto 最优解优于 MOPSO 算法求得 Pareto 最优解。

由于题目并未给出对于磁通损耗以及传输磁能的偏好程度，对于上述求得的 Pareto 前沿，需要结合偏好程度进一步获取对应的最优解。

8.3 本章小结

因此，本文以温度、频率、波形、磁通密度峰值及磁芯材料作为决策变量，以问题四提出的磁芯损耗预测模型和传输磁能作为目标函数，通过分析决策变量的约束范围，最终构建了一个多目标优化问题，旨在确定在什么温度、频率、波形、磁通密度峰值及磁芯材料条件下能够实现最小的磁芯损耗和最大的传输磁能。

本文首先通过对数据和问题分析建立起多目标优化问题模型，针对多目标优化问题，通过加权转化为单目标优化问题，通过智能优化算法两种方式对模型进行求解，针对加权转化后的单目标优化问题，采用遗传算法和粒子群算法求取最优解。最终结果表明，遗传算法求得的最优解优于粒子群算法求得的最优解，最终多目标转化单目标优化方式求得的最优解为：磁芯材料二，正弦波，温度 90°C，频率 79680Hz，磁通密度峰值 0.021，对应的磁芯损耗为 426.3647，最大传输磁能为 1673.28；针对智能优化算法，本文采用 NSGA-II 算法和 MOPSO 算法两种优化算法分别求取了多目标优化问题的 Pareto 最优解，绘制了 Pareto 前沿，并对两个算法求取的 Pareto 最优解进行了评价，最终得到 NSGA-II 算法求得 Pareto 最优解优于 MOPSO 算法求得 Pareto 最优解。

九、模型评价与推广

9.1 模型的优点

(1)本文对于每个问题都采用了多个不同的模型进行预测和评估，并且通过对比分析各模型的预测效果来选择表现出较为优秀的模型。比如，采用了粒子群算法、模拟退火算法、最小二乘法进行斯坦麦茨方程参数寻优。

(2)本文对现有的模型进行了改进，例如在粒子群算法中加入了动态参数调整机制，提出了一种针对五种特征的特征权重提取的双向 LSTM 磁芯损耗预测模型。

9.2 模型的缺点

(1)由于计算能力与时间的限制，本文在各个模型训练中没有对超参数进行寻优，只是简单的进行网格搜索，不能表明当前结果是模型的最好结果，但能证明模型有效性。

(2)由于数据限制，对深度模型来说当前数据集仍不足，若使用更多数据集，例如更多材料，更多温度等进行对比，模型将更具说服力。

9.3 模型改进与推广

(1)在后续研究中，可以采用更多的特征进行磁芯损耗的预测，更多的特征可以提高模型的预测精度。

(2)本文在模型研究中，选择了深度学习模型并且验证了其泛化能力，在磁芯损耗预测工作中可以考虑本文建立的模型。

(3)在模型推广方面，可以进一步推广应用范围，例如对其他产品或工件的损耗进行预测。

十、参考文献

- [1]S. S. Ben-Yaakov, "SPICE simulation of ferrite core losses and hot spot temperature estimation," 2017 24th IEEE International Conference on Electronics, Circuits and Systems (ICECS), Batumi, Georgia, 2017, pp. 107-110.
- [2]M. H. F. Lim and J. D. van Wyk, "Applying the Steinmetz Model to Small Cores in LTCC Ferrite for Integration Applications," 2009 Twenty-Fourth Annual IEEE Applied Power Electronics Conference and Exposition, Washington, DC, USA, 2009, pp. 1027-1033.
- [3]Srinivas N, Deb K. Multiobjective optimization using nondominated sorting in genetic algorithms[J]. Evolutionary computation, 1994, 2(3): 221-248.
- [4] Holland J H. Genetic algorithms[J]. Scientific american, 1992, 267(1): 66-73.
- [5]冯珂豪.高频磁芯兆赫兹频段损耗测试系统研究[D].杭州电子科技大学,2024.DOI:10.27075/d.cnki.ghzdc.2024.000070. [6] Iglovikov V , Shvets A .TernausNet: U-Net with VGG11 Encoder Pre-Trained on ImageNet for Image Segmentation[J]. 2018.
- [6]赵磊,刘成成,汪友华.谐波激励下软磁复合材料磁芯损耗的有限元计算[J].兵器材料科学与工程,2023,46(03):13-21.DOI:10.14024/j.cnki.1004-244x.20230519.005.
- [7] Y. Zhou, X. Song and M. Zhou, "Supply Chain Fraud Prediction Based On XGBoost Method," 2021 IEEE 2nd International Conference on Big Data, Artificial Intelligence and Internet of Things Engineering (ICBAIE), Nanchang, China, 2021, pp. 539-542.
- [8] A. Diaaeldin and M. Zaher, "Enhancing Road Safety: Leveraging CNN-LSTM and Bi-LSTM Models for Advanced Driver Behavior Detection," 2024 Intelligent Methods, Systems, and Applications (IMSA), Giza, Egypt, 2024, pp. 416-422.
- [9]W. Jia and Z. He, "Outlier data Mining of large Data Sets relying on fast decomposition simulated annealing algorithm," 2020 International Conference on Robots & Intelligent System (ICRIS), Sanya, China, 2020, pp. 679-682.

附 录：Python 代码

程序 1	动态参数粒子群算法
<pre> class PSO_Gaijin(SkoBase): def __init__(self, func, n_dim=None, pop=40, max_iter=150, lb=-1e5, ub=1e5, w=0.8, c1=0.5, c2=0.5, constraint_eq=tuple(), constraint_ueq=tuple(), verbose=False , dim=None): n_dim = n_dim or dim # support the earlier version self.func = func_transformer(func) self.w = w # inertia self.cp, self.cg = c1, c2 # parameters to control personal best, global best respectively self.pop = pop # number of particles self.n_dim = n_dim # dimension of particles, which is the number of variables of func self.max_iter = max_iter # max iter self.verbose = verbose # print the result of each iter or not self.lb, self.ub = np.array(lb) * np.ones(self.n_dim), np.array(ub) * np.ones(self.n_dim) assert self.n_dim == len(self.lb) == len(self.ub), 'dim == len(lb) == len(ub) is not True' assert np.all(self.ub > self.lb), 'upper-bound must be greater than lower-bound' self.has_constraint = bool(constraint_ueq) self.constraint_ueq = constraint_ueq </pre>	

```

self.is_feasible = np.array([True] * pop)

self.X = np.random.uniform(low=self.lb, high=self.ub,
size=(self.pop, self.n_dim))

v_high = self.ub - self.lb

self.V = np.random.uniform(low=-v_high, high=v_high,
size=(self.pop, self.n_dim)) # speed of particles

self.Y = self.cal_y() # y = f(x) for all particles

self.pbest_x = self.X.copy() # personal best location of every
particle in history

self.pbest_y = np.array([[np.inf]] * pop) # best image of every
particle in history

self.gbest_x = self.pbest_x.mean(axis=0).reshape(1, -1) # global
best location for all particles

self.gbest_y = np.inf # global best y for all particles

self.gbest_y_hist = [] # gbest_y of every iteration

self.update_gbest()

# record verbose values
self.record_mode = False

self.record_value = {'X': [], 'V': [], 'Y': []}

self.best_x, self.best_y = self.gbest_x, self.gbest_y # history
reasons, will be deprecated

def check_constraint(self, x):

    # gather all unequal constraint functions

    for constraint_func in self.constraint_ueq:

        if constraint_func(x) > 0:

            return False

    return True

```

```

def update_V(self):
    r1 = np.random.rand(self.pop, self.n_dim)
    r2 = np.random.rand(self.pop, self.n_dim)
    self.V = self.w * self.V + \
        self.cp * r1 * (self.pbest_x - self.X) + \
        self.cg * r2 * (self.gbest_x - self.X)

def update_X(self):
    self.X = self.X + self.V
    self.X = np.clip(self.X, self.lb, self.ub)

def cal_y(self):
    # calculate y for every x in X
    self.Y = self.func(self.X).reshape(-1, 1)
    return self.Y

def update_pbest(self):
    self.need_update = self.pbest_y > self.Y
    for idx, x in enumerate(self.X):
        if self.need_update[idx]:
            self.need_update[idx] = self.check_constraint(x)

    self.pbest_x = np.where(self.need_update, self.X, self.pbest_x)
    self.pbest_y = np.where(self.need_update, self.Y, self.pbest_y)

def update_gbest(self):

```

```

idx_min = self.pbest_y.argmax()

if self.gbest_y > self.pbest_y[idx_min]:
    self.gbest_x = self.X[idx_min, :].copy()
    self.gbest_y = self.pbest_y[idx_min]

def recorder(self):
    if not self.record_mode:
        return

    self.record_value['X'].append(self.X)
    self.record_value['V'].append(self.V)
    self.record_value['Y'].append(self.Y)

def run(self, max_iter=None, precision=None, N=20):
    self.max_iter = max_iter or self.max_iter
    c = 0
    for iter_num in range(self.max_iter):
        self.update_V()
        self.recorder()
        self.update_X()
        self.cal_y()
        self.update_pbest()
        self.update_gbest()
        self.w = 0.9 - (0.5 * iter_num / self.max_iter) # Decrease
inertia
        self.cp = 0.5 + (1.0 * iter_num / self.max_iter) # Increase
cognitive component
        self.cg = 0.5 + (1.0 * iter_num / self.max_iter) # Increase
social component

```

```

        if precision is not None:

            tor_iter = np.amax(self.pbest_y) - np.amin(self.pbest_y)

            if tor_iter < precision:

                c = c + 1

                if c > N:

                    break

            else:

                c = 0

        if self.verbose:

            print('Iter: {}, Best fit: {} at {}'.format(iter_num,
self.gbest_y, self.gbest_x))

        self.gbest_y_hist.append(self.gbest_y)

        self.best_x, self.best_y = self.gbest_x, self.gbest_y

        return self.best_x, self.best_y

```

程序 2	特征权重提取的Bi-LSTM网络
<pre> x_train_tensor = torch.FloatTensor(x_train).to(device) x_label_tensor = torch.FloatTensor(x_label).to(device) v_train_tensor = torch.FloatTensor(v_train).to(device) v_label_tensor = torch.FloatTensor(v_label).to(device) y_train_tensor = torch.FloatTensor(y_train).to(device) y_label_tensor = torch.FloatTensor(y_label).to(device) # 创建数据集和数据加载器 train_dataset = TensorDataset(x_train_tensor, x_label_tensor) train_loader = DataLoader(train_dataset, batch_size=64, shuffle=True) # 定义一维卷积模型 class CNNModel(nn.Module): def __init__(self, input_size): super(CNNModel, self).__init__() self.conv1 = nn.Conv1d(in_channels=1, out_channels=64, kernel_size=3, stride=1, padding=1) self.conv2 = nn.Conv1d(in_channels=64, out_channels=128, kernel_size=3, stride=1, padding=1) self.conv3 = nn.Conv1d(in_channels=128, out_channels=256, kernel_size=3, stride=1, padding=1) # 计算全连接层输入的特征数量 self.fc_input_size = 256 * input_size # 32 是最后一层卷积输出 的通道数 self.fc = nn.Linear(self.fc_input_size, 1) def forward(self, x): </pre>	

```

x = x.unsqueeze(1) # 增加通道维度 (batch_size, 1, seq_length)

x = self.conv1(x) # 通过第一个卷积层
x = nn.ReLU()(x) # ReLU 激活
x = self.conv2(x) # 通过第二个卷积层
x = nn.ReLU()(x) # ReLU 激活
x = self.conv3(x) # 通过第二个卷积层
x = nn.ReLU()(x) # ReLU 激活
x = x.view(x.size(0), -1) # flatten the output
output = self.fc(x) # 全连接层
return output

# 实例化模型、损失函数和优化器，并移动模型到 GPU
input_size = 1033 # 输入特征的数量
best_v_loss = float('inf') # 初始化最佳验证损失
best_model_path = 'best_cnnmodel.pth' # 模型保存路径
model = CNNModel(input_size).to(device)
criterion = nn.MSELoss()
optimizer = optim.Adam(model.parameters(), lr=0.00005)
def r2_loss(y_true, y_pred):
    """计算 R2 损失"""
    ss_total = ((y_true - torch.mean(y_true)) ** 2).sum()
    ss_residual = ((y_true - y_pred) ** 2).sum()
    r2 = 1 - (ss_residual / ss_total)
    return 1 - r2 # 返回 1 - R2 作为损失

# 在训练模型之前，添加一个列表来存储训练损失

```

```

train_losses = []

# 训练模型

num_epochs = 500

for epoch in range(num_epochs):

    model.train()

    epoch_loss = 0.0 # 初始化每个 epoch 的损失

    for batch_x, batch_y in train_loader:

        optimizer.zero_grad() # 清空梯度

        outputs = model(batch_x) # 前向传播

        loss = r2_loss(batch_y.view(-1, 1), outputs) # 计算损失

        loss.backward() # 反向传播

        optimizer.step() # 更新参数

        epoch_loss += loss.item() # 累加损失

    avg_epoch_loss = epoch_loss / len(train_loader) # 计算平均损失

    train_losses.append(1 - avg_epoch_loss) # 存储 R² 损失

    print(f'Epoch [{epoch+1}/{num_epochs}], Loss: {1 - avg_epoch_loss:.8f}')

# 验证模型

model.eval()

with torch.no_grad():

    v_outputs = model(v_train_tensor)

    v_loss = r2_loss(v_label_tensor.view(-1, 1), v_outputs)

# 保存最佳模型

```

```

        if v_loss < best_v_loss:
            best_v_loss = v_loss
            torch.save(model.state_dict(), best_model_path)
            print(f'Saved Best Model at Epoch [{epoch + 1}] with
Validation Loss: {1 - v_loss.item():.8f}')

# 所有 epoch 运行完后，加载最佳模型进行评估
model.load_state_dict(torch.load(best_model_path))
model.eval() # 设置为评估模式

# 预测和计算指标（与之前相同）
with torch.no_grad():
    v_outputs = model(y_train_tensor)

# 转换为 NumPy 数组并计算指标（与之前相同）
v_outputs = v_outputs.cpu().numpy()
v_label = y_label_tensor.cpu().numpy()
v_label = np.exp(v_label)
v_outputs = np.exp(v_outputs)

r2 = r2_score(v_label, v_outputs)
mape = np.mean(np.abs((v_label - v_outputs) / v_label)) * 100
rmse = np.sqrt(mean_squared_error(v_label, v_outputs))

```

程序 3	传统CNN网络
<pre> class CNNModel(nn.Module): def __init__(self, input_size): super(CNNModel, self).__init__() self.conv1 = nn.Conv1d(in_channels=1, out_channels=64, kernel_size=3, stride=1, padding=1) self.conv2 = nn.Conv1d(in_channels=64, out_channels=128, kernel_size=3, stride=1, padding=1) self.conv3 = nn.Conv1d(in_channels=128, out_channels=256, kernel_size=3, stride=1, padding=1) # 计算全连接层输入的特征数量 self.fc_input_size = 256 * input_size # 32 是最后一层卷积 输出的通道数 self.fc = nn.Linear(self.fc_input_size, 1) def forward(self, x): x = x.unsqueeze(1) # 增加通道维度 (batch_size, 1, seq_length) x = self.conv1(x) # 通过第一个卷积层 x = nn.ReLU()(x) # ReLU 激活 x = self.conv2(x) # 通过第二个卷积层 x = nn.ReLU()(x) # ReLU 激活 x = self.conv3(x) # 通过第二个卷积层 x = nn.ReLU()(x) # ReLU 激活 x = x.view(x.size(0), -1) # flatten the output output = self.fc(x) # 全连接层 </pre>	

```

        return output

# 实例化模型、损失函数和优化器，并移动模型到 GPU
input_size = 10 # 输入特征的数量

best_v_loss = float('inf') # 初始化最佳验证损失

best_model_path = 'best_cnnmodel10.pth' # 模型保存路径

model = CNNModel(input_size).to(device)

criterion = nn.MSELoss()

optimizer = optim.Adam(model.parameters(), lr=0.00005)

def r2_loss(y_true, y_pred):
    """计算 R2 损失"""
    ss_total = ((y_true - torch.mean(y_true)) ** 2).sum()
    ss_residual = ((y_true - y_pred) ** 2).sum()
    r2 = 1 - (ss_residual / ss_total)

    return 1 - r2 # 返回 1 - R2 作为损失

## 训练模型

num_epochs = 500

for epoch in range(num_epochs):

    model.train()

    for batch_x, batch_y in train_loader:

        # print("Original Batch X Shape:", batch_x.shape) # 打印原始形状

        # batch_x = batch_x.unsqueeze(1) # 增加通道维度
        # (batch_size, 1, seq_length)

        # print("Modified Batch X Shape:", batch_x.shape) # 打印修改后的形状

        optimizer.zero_grad() # 清空梯度

```

```

        outputs = model(batch_x)      # 前向传播

        # loss = criterion(outputs, batch_y.view(-1, 1)) # 计算损失
        loss = r2_loss(batch_y.view(-1, 1), outputs)

        loss.backward()                # 反向传播

        optimizer.step()               # 更新参数

    print(f'Epoch [{epoch+1}/{num_epochs}], Loss: {1-
loss.item():.8f}')

# 验证模型
model.eval()
with torch.no_grad():
    v_outputs = model(v_train_tensor)

# 计算验证集的 R2 损失
v_loss = r2_loss(v_label_tensor.view(-1, 1), v_outputs)

# 保存最佳模型
if v_loss < best_v_loss:
    best_v_loss = v_loss

    torch.save(model.state_dict(), best_model_path) # 保存
模型参数

    print(f'Saved Best Model at Epoch [{epoch + 1}] with
Validation Loss: {1 - v_loss.item():.8f}')

# 转换为 NumPy 数组
v_outputs = v_outputs.cpu().numpy()
v_label = v_label_tensor.cpu().numpy()

```

程序 4	第四问数据预处理
<pre> # 加载保存的数组 materials = np.zeros((12400,4)) materials[0:3400] = [1, 0, 0, 0] materials[3400:6400] = [0, 1, 0, 0] materials[6400:9600] = [0, 0, 1, 0] materials[9600:12400] = [0, 0, 0, 1] temperatures = np.load('temperatures.npy') frequencies = np.load('frequencies.npy') core_losses = np.load('core_losses.npy') flux_densities = np.load('flux_densities.npy') # plot1(flux_densities[0]) waveforms = np.load('waveforms.npy', allow_pickle=True) frequencies = f_normalization(frequencies) temperatures = t_normalization(temperatures) # 类别的开始位置 category_starts = [0, 3400, 6400, 9600] category_sizes = [3400, 3000, 3200, 2800] # 每个类别的大小 # 初始化训练集和测试集的列表 train_indices = [] test_indices = [] # 设定随机种子 np.random.seed(42) # 按类别划分 for start, size in zip(category_starts, category_sizes): </pre>	

```

indices = np.arange(start, start + size)

np.random.shuffle(indices) # 打乱顺序

split_index = int(size * 0.8) # 80% 的索引

train_indices.extend(indices[:split_index]) # 训练集索引

test_indices.extend(indices[split_index:]) # 测试集索引

# 转换为 NumPy 数组
train_indices = np.array(train_indices)
test_indices = np.array(test_indices)
train_number = int(12400*0.8)
test_number = int(12400*0.2)
print(train_number)

# 获取训练集和测试集数据

temperatures_train = temperatures[train_indices].reshape(train_number,1) # 确保形状一致
temperatures_test = temperatures[test_indices].reshape(test_number,1)

frequencies_train = frequencies[train_indices].reshape(train_number,1)
frequencies_test = frequencies[test_indices].reshape(test_number,1)

materials_train = materials[train_indices].reshape(train_number,4)
materials_test = materials[test_indices].reshape(test_number,4)

waveforms_train = waveforms[train_indices].reshape(train_number,3)
waveforms_test = waveforms[test_indices].reshape(test_number, 3)

```

```
flux_densities_train =  
flux_densities[train_indices].reshape(train_number, 1024)  
  
flux_densities_test = flux_densities[test_indices].reshape(test_number,  
1024)  
  
plot1(flux_densities_train[8],title='梯形波')  
  
core_losses_train = core_losses[train_indices] # 标签  
core_losses_test = core_losses[test_indices]  
  
train_label = np.log(core_losses_train)  
test_label = np.log(core_losses_test)
```